KYANOS HOTELS & RESORTS

Unified Data & AI Platform on Microsoft Fabric

Solution documentation – architecture, data model, DAX, AI and build process

Item	Value
Document	Solution documentation · version 1.0
Date	7 October 2026
Author	Vangelis Misirlis
Workspace	"Kyanos Hotels - Analytics & AI Demo" (tested on F4 and F8 capacities)
Purpose	Reference demo for hotel groups: real-time PMS integration, unified analytics, AI forecasting and data agents
Data	Fully synthetic. Kyanos Hotels & Resorts, its hotels, guests and partners are fictional.

Contents

1. Executive summary
2. Solution architecture
3. Data sources and synthetic data
4. Ingestion: PMS mirroring, shortcut and file landing
5. Lakehouse design and data lineage
6. Notebooks and transformation logic
7. Semantic model and table relationships
8. DAX measure catalog
9. DAX queries (validation and analysis)
10. Power BI report
11. AI demand forecast
12. Fabric data agent
13. Fabric IQ ontology
14. Live PMS simulator and demo operations
15. Build and deployment process
16. Issues encountered and resolutions
17. Important considerations
18. Appendices: item inventory, column dictionary, code excerpts, glossary

1. Executive summary

This document describes an end-to-end Microsoft Fabric solution built for a hotel group. It shows how a group that today runs an on-premises PMS and SQL Server data warehouse with a nightly batch (the 05:00–08:00 window) can move to **one copy of the data in OneLake**, **near real-time reporting**, **AI forecasting** and **natural-language access**, without re-platforming its operational systems.

1.1 What was built

- **Operational source** – `KyanosPMS`, a Fabric SQL database standing in for an on-premises PMS / SQL Server, holding 24 hotels, 117 room types, 255,848 guests and 483K reservations. It is replicated to OneLake automatically (Mirroring).
- **Unified lakehouse** – `KyanosLakehouse` (schema-enabled) combining the PMS (zero-copy shortcut), Salesforce (Privilege Club, service cases), SAP (monthly hotel costs) and airline seat capacity, refined into a gold star schema by PySpark notebooks.
- **Governed semantic model** – `Kyanos Hotels Model`: Direct Lake on OneLake, 11 tables, 15 relationships, 53 DAX measures (Occupancy, ADR, RevPAR, GOP, pace, forecast, live KPIs) and 2 row-level-security roles.
- **Power BI report** – 5 pages with a custom Aegean theme: Executive Overview, Hotel Scorecard, Channels & Guests, 2027 Demand Outlook (AI) and Live PMS Feed.
- **AI demand forecast** – a gradient-boosting booking-pace model (MLflow-tracked and registered) that forecasts 2027 occupancy per hotel-night from booking pace and published airline seat capacity.
- **Fabric data agent** – "Kyanos Revenue & Guest Assistant" answering Greek and English questions over the same governed model.
- **Fabric IQ ontology** – six business entities (Hotel, Airport, VIP guest, Upcoming stay, Open case, Tour operator) with five relationships.
- **Live simulator** – a Python process writing new bookings into the PMS; they appear in Power BI in about **13 seconds**.

1.2 Headline results in the demo data

KPI (Jan–Sep closed months)	2025	2026	Change
Total revenue	€362,577,449	€384,982,838	6.2%
Occupancy	83.2%	84.3%	+1.1 pts
ADR	€300.02	€318.45	6.1%
RevPAR	€249.56	€268.46	7.6%
GOP margin	39.5%	40.2%	+0.7 pts
Direct booking share	34.5%	38.3%	+3.8 pts

2027 outlook (AI forecast, Jan–Sep)	Value
Forecast occupancy 2027	86.0%
Actual occupancy 2026 (same weekdays)	84.4%
Already on the books (snapshot 6 Oct 2026)	7.1%

2027 outlook (AI forecast, Jan–Sep)	Value
Forecast room revenue	€319,509,787 (5.4% vs 2026)
Model accuracy (holdout MAE)	3.31 occupancy points vs 4.09 for "same as last year"

1.3 Key design decisions

- **Mirror, don't extract.** Operational SQL data lands in OneLake continuously; the nightly copy and the 05:00–08:00 window disappear.
- **Zero-copy unification.** The PMS is exposed in the lakehouse through a schema-level OneLake shortcut, so there is one physical copy.
- **Direct Lake semantic model.** Power BI reads Delta tables directly – no import refresh, and new commits are framed automatically.
- **One set of KPI definitions.** Report, Copilot and the data agent all use the same 53 measures, so numbers match everywhere.
- **AI on the same platform.** Forecasting runs in a notebook on the same capacity and writes back to gold; MLflow tracks and registers the model.

2. Solution architecture

The solution follows a five-layer flow: **Sources** → **Ingestion** → **OneLake** → **Analytics & AI** → **Consumption**, all inside one Fabric workspace on an F8 capacity. The workspace is organised into folders that mirror these layers.

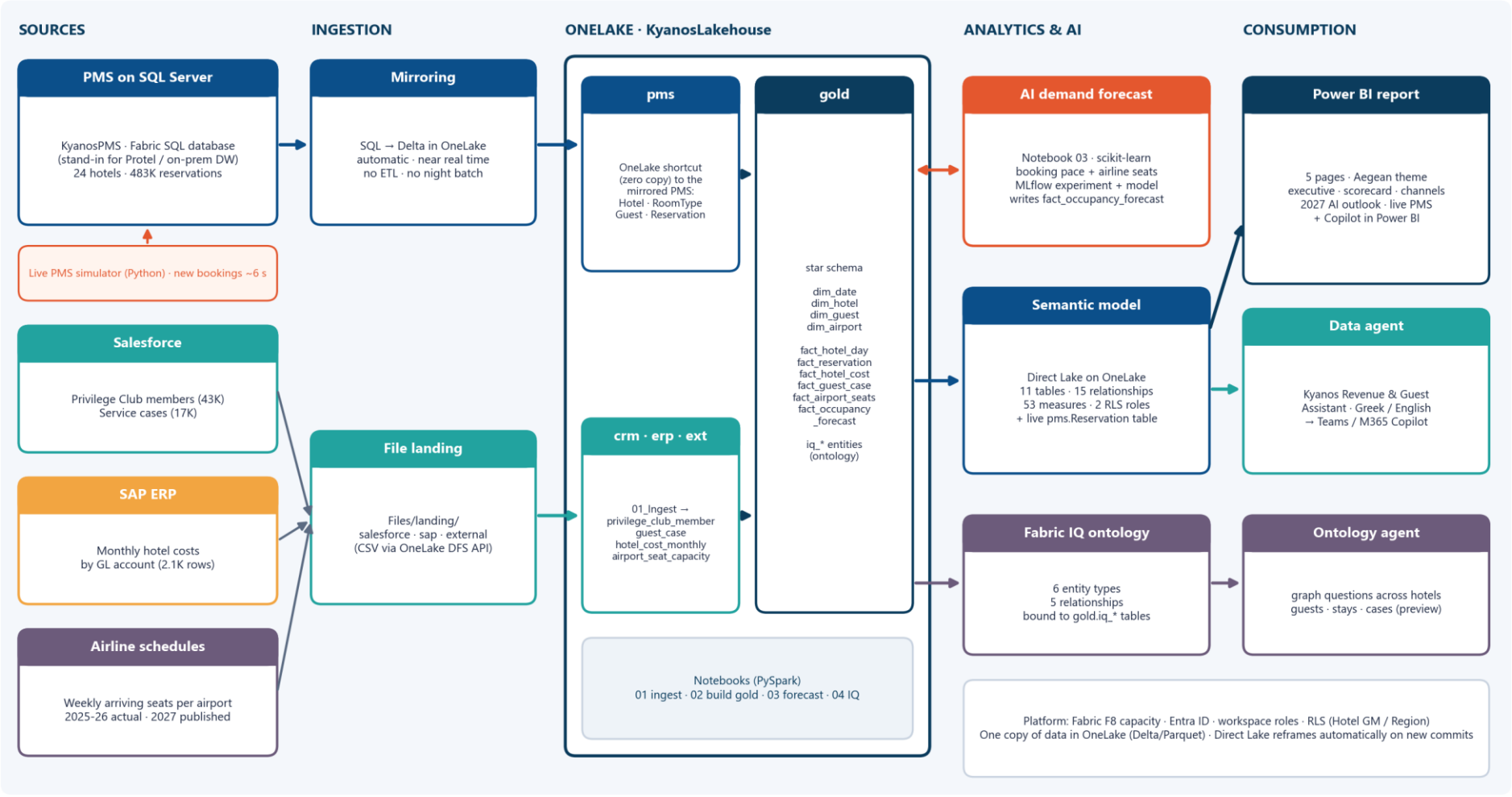


Figure 1 – Solution architecture

2.1 Layer-by-layer description

Layer	Fabric item(s)	Role	Key facts
Sources	KyanosPMS (SQL database); CSV extracts	Operational PMS; Salesforce, SAP and airline data	PMS is the system of record for stays and revenue; others arrive as files
Ingestion	Mirroring; OneLake shortcut; OneLake DFS API	Continuous replication of SQL tables; file landing	Mirrored Delta committed within seconds of each insert
OneLake	KyanosLakehouse (schemas pms, crm, erp, ext, gold)	Single copy of data; medallion refinement	24 tables incl. 10 gold model tables and 6 iq_* entities
Transformation	Notebooks 01, 02, 04 (PySpark)	Typing, conformance, data quality, star schema	Run via the Fabric Jobs API; ~1–3 min each on F8
AI	Notebook 03; MLflow experiment + model	Occupancy forecast for Jan–Sep 2027	HistGradientBoosting with monotonic constraints
Semantic layer	Kyanos Hotels Model (Direct Lake)	Governed KPIs, relationships, RLS	11 tables, 15 relationships, 53 measures, 2 roles
Consumption	Power BI report; data agent; ontology	Dashboards, Copilot, natural-language Q&A, graph	Greek and English questions; Teams / M365 Copilot publishable

2.2 Design principles

- **One copy of data:** Delta/Parquet in OneLake is read by Spark, SQL, Power BI (Direct Lake), the data agent and the ontology.
- **Near real time where it matters:** the PMS is mirrored; the semantic model includes the live `pms.Reservation` table for the Live PMS page.
- **Schema-enabled lakehouse:** schemas separate sources (`pms`, `crm`, `erp`, `ext`) from serving tables (`gold`).
- **Governed metrics:** every KPI is a model measure with description and display folder; visuals and agents never recompute KPIs.
- **Security by design:** RLS anchored on `dim_hotel` (per-hotel GM and per-region roles); workspace roles control authoring.
- **Everything as code:** every item is deployed through Fabric REST APIs from Python, making the demo repeatable in any tenant.

2.3 Workspace organisation

Folder	Items
1 Sources PMS on SQL	KyanosPMS (SQL database + its SQL analytics endpoint)
2 OneLake unified data	KyanosLakehouse, 01_Ingest_Salesforce_SAP_Airports, 02_Build_Gold_Model
3 AI forecasting	03_AI_Demand_Forecast_2027, MLflow experiment and registered model kyanos-occupancy-forecast
4 Power BI	Kyanos Hotels Model (semantic model), Kyanos Hotels - Commercial Performance (report)
5 Agents and Fabric IQ	Kyanos Revenue & Guest Assistant (data agent), KyanosHotelsOntology (+ its lakehouse and graph model), 04_Fabric_IQ_Ontology_Entities

3. Data sources and synthetic data

All data is generated by `src/gen_data.py` with a fixed random seed, so the dataset is reproducible. The **snapshot date is 6 October 2026**: stays before it are realised (checked-out or in-house), later stays are on-the-books (OTB) business.

3.1 Hotel portfolio

Code	Hotel	Region	Airport	Concept	Rooms	Season
KYN-ELO	Kyanos Elounda Palace	Crete	HER	Luxury Resort	280	04-01 → 11-05
KYN-AGN	Kyanos Agios Nikolaos Bay	Crete	HER	Luxury Resort	220	04-05 → 10-31
KYN-HER	Kyanos Heraklion City	Crete	HER	City Hotel	160	Year-round
KYN-RET	Kyanos Rethymno Mare	Crete	CHQ	Family Resort	380	04-10 → 10-31
KYN-CHA	Kyanos Chania Kalyves	Crete	CHQ	Family Resort	240	04-15 → 10-25
KYN-MAL	Kyanos Malia Riviera	Crete	HER	All-Inclusive Resort	420	04-10 → 10-31
KYN-IXI	Kyanos Rhodes Ixia Grand	Rhodes	RHO	Luxury Resort	360	04-01 → 11-01
KYN-LIN	Kyanos Lindos Blue	Rhodes	RHO	Adults-Only Resort	180	04-10 → 10-31
KYN-FAL	Kyanos Faliraki Family	Rhodes	RHO	All-Inclusive Resort	450	04-15 → 10-31
KYN-MAR	Kyanos Kos Marmari	Kos	KGS	All-Inclusive Resort	400	04-20 → 10-25
KYN-PSA	Kyanos Kos Psalidi Suites	Kos	KGS	Luxury Resort	200	04-20 → 10-31
KYN-DAS	Kyanos Corfu Dassia	Corfu	CFU	Luxury Resort	320	04-25 → 10-25
KYN-KAV	Kyanos Kavos Lagoon	Corfu	CFU	All-Inclusive Resort	260	05-01 → 10-15
KYN-PAL	Kyanos Paleokastritsa	Corfu	CFU	Family Resort	150	04-25 → 10-25
KYN-ORN	Kyanos Mykonos Ornos	Cyclades	JMK	Luxury Resort	120	04-25 → 10-31
KYN-CAL	Kyanos Santorini Caldera	Cyclades	JTR	Adults-Only Resort	90	04-01 → 11-05
KYN-KAS	Kyanos Kassandra Palace	Halkidiki	SKG	Luxury Resort	340	05-01 → 10-15
KYN-SIT	Kyanos Sithonia Beach	Halkidiki	SKG	Family Resort	300	05-01 → 10-10
KYN-THE	Kyanos Thessaloniki Center	Macedonia	SKG	City Hotel	180	Year-round
KYN-OLY	Kyanos Olympia Riviera	Peloponnese	GPA	Family Resort	380	04-01 → 10-31
KYN-MES	Kyanos Messinia Bay	Peloponnese	KLX	Luxury Resort	260	04-01 → 11-01
KYN-ACR	Kyanos Athens Acropolis	Attica	ATH	City Hotel	210	Year-round
KYN-VOU	Kyanos Vouliagmeni Riviera	Attica	ATH	Luxury Resort	150	03-15 → 11-30
KYN-ZAK	Kyanos Zakynthos Laganas	Zakynthos	ZTH	All-Inclusive Resort	260	05-01 → 10-20

3.2 Data volumes

Dataset	System	Rows	Notes
pms.Hotel / pms.RoomType	PMS (SQL)	24 / 117	Resorts: 5 room types; city hotels: 4
pms.Guest	PMS (SQL)	255,848	Fictional names, emails at example.com, 20 source markets
pms.Reservation	PMS (SQL)	484,782	Arrivals 2025-01-01 → 2027; includes cancellations, no-shows, in-house, OTB; plus live simulator rows
privilege_club_members.csv	Salesforce	43,250	Tier Blue/Silver/Gold/Platinum by lifetime nights; 99 duplicate member IDs (deliberate data-quality issue)
guest_cases.csv	Salesforce	17,379	Complaints, requests, compliments with CSAT 1–5

Dataset	System	Rows	Notes
hotel_costs_monthly.csv	SAP	2,148	6 cost categories by GL account per hotel-month
airport_seat_capacity_weekly.csv	External	1,884	12 airports; weekly arriving seats; 2027 = published schedules

3.3 Generation logic

- **Inventory-consistent occupancy:** for each hotel, room type and day a target occupancy is drawn (monthly seasonality × weekday × hotel performance × year factor × noise). Arrivals are created until the target is met, with length of stay truncated at full nights or hotel closure – occupancy can never exceed capacity.
- **Seasonality:** resorts peak in July–August (96–98%) and close between November and March/April; city hotels run all year with a weekday/weekend pattern.
- **Channels and lead times:** channel mix depends on concept (tour operators dominate all-inclusive resorts, OTAs and corporate dominate city hotels). Lead times are log-normal per channel (tour operators ≈190 days, OTAs ≈40 days).
- **Prices:** ADR = base rate × room-type multiplier × monthly season factor × channel factor × board factor × year factor (+6.5% in 2026) × noise.
- **Cancellations and snapshot:** cancellation rates per channel (Booking.com 24%, tour operators 7%). Only bookings made on or before the snapshot are kept; future cancellations stay as Confirmed.
- **Extras revenue:** F&B and other spend per adult-night depends on board type and hotel concept, realised only for stayed nights.
- **SAP costs:** payroll per room, F&B cost of sales (38%), energy (+12% in 2026), OTA/TO commissions, maintenance and marketing – producing a ~40% GOP margin.

3.4 Built-in business stories

Story	How it appears	Where to show it
Airlines cut seats to Kos and Zakynthos in 2026	KGS −13%, ZTH −10% seats; RevPAR −6% to −8%; rates held flat	Executive Overview, Hotel Scorecard, data agent
Direct booking strategy is working	≈12% of OTA demand shifts to direct web in 2026; direct share 34.5% → 38.3%	Channels & Guests
Air-conditioning issue at Kos Marmari	35% of its 2026 cases are A/C complaints (151 vs 37 for the next category)	Hotel Scorecard (click the hotel)
Summer 2027 seat growth	Araxos +15%, Chania +12%, Kalamata +11% → forecast up; Kos still declining	2027 Demand Outlook (AI)
Data-quality issue between CRM and PMS	99 duplicate Privilege Club IDs	Notebook 02 (de-duplication)

4. Ingestion: PMS mirroring, shortcut and file landing

4.1 PMS database (KyanosPMS)

`KyanosPMS` is a **Fabric SQL database** (SQL Server engine). It plays the role of the customer's on-premises Protel/SQL Server. All tables have primary keys – a prerequisite for mirroring from SQL Server 2016–2022.

4.1.1 pms.Hotel

Column definition
HotelId INT NOT NULL PRIMARY KEY
HotelCode VARCHAR(10) NOT NULL
HotelName NVARCHAR(80) NOT NULL
Destination NVARCHAR(40)
Region NVARCHAR(40)
Airport CHAR(3)
Stars TINYINT
Concept NVARCHAR(40)
Rooms INT
Latitude DECIMAL(9,6)
Longitude DECIMAL(9,6)
OpenMMDD CHAR(5)
CloseMMDD CHAR(5)
IsSeasonal BIT
GeneralManager NVARCHAR(60)
GMEmail NVARCHAR(80)

4.1.2 pms.RoomType

Column definition
RoomTypeId INT NOT NULL PRIMARY KEY
HotelId INT NOT NULL
RoomTypeCode VARCHAR(6)
RoomTypeName NVARCHAR(60)
RoomCount INT
RateMultiplier DECIMAL(5,2)

4.1.3 pms.Guest

Column definition
GuestId INT NOT NULL PRIMARY KEY
FirstName NVARCHAR(40)
LastName NVARCHAR(40)
Email NVARCHAR(120)
Country CHAR(2)
CountryName NVARCHAR(40)
Language NVARCHAR(20)
LoyaltyNumber VARCHAR(12) NULL
CreatedAt DATETIME2(3)

4.1.4 pms.Reservation

Column definition
`ReservationId BIGINT NOT NULL PRIMARY KEY`
`ConfirmationNo VARCHAR(12) NOT NULL`
`HotelId INT NOT NULL`
`RoomTypeId INT NOT NULL`
`GuestId INT NOT NULL`
`BookingDate DATE NOT NULL`
`ArrivalDate DATE NOT NULL`
`DepartureDate DATE NOT NULL`
`Nights INT NOT NULL`
`Adults TINYINT`
`Children TINYINT`
`Channel NVARCHAR(30)`
`MarketSegment NVARCHAR(30)`
`TourOperator NVARCHAR(40) NULL`
`BoardType NVARCHAR(20)`
`RateAmount DECIMAL(10,2)`
`RoomRevenue DECIMAL(12,2)`
`ExtrasRevenue DECIMAL(12,2)`
`Status NVARCHAR(15)`
`CancelDate DATE NULL`
`CreatedAt DATETIME2(3)`
`ModifiedAt DATETIME2(3)`

4.2 Loading the PMS

- `src/load_pms.py` creates the `pms` schema and tables, then loads hotels, room types, 255,848 guests and 482,961 reservations.
- Connectivity uses **mssql-python** (bundled driver, no ODBC install) with an **Entra access token** from the Azure CLI passed as `SQL_COPT_SS_ACCESS_TOKEN` (attribute 1256).
- Throughput: ~2,000–6,700 rows/s with `executemany` (bulk copy opened a second connection without the token, so the loader falls back automatically). Total load time ≈ 5 min 40 s.
- An index `IX_Res_Booking (BookingDate) INCLUDE (HotelId, Status)` supports the simulator and booking-date queries.

4.3 Mirroring to OneLake

A Fabric SQL database replicates every table to OneLake as Delta automatically. Measured: a new Delta log version appeared **within seconds** of an insert, and the Direct Lake model showed **12 new bookings 13 seconds** after they were written.

> **Customer equivalent (SQL Server 2022 on-premises)**
>
> Use the **Mirrored database** item for SQL Server through the on-premises data gateway already used for Power BI. Mirroring SQL Server 2016–2022 relies on **CDC**, needs a **primary key** on every mirrored table, runs from the **primary replica** of an availability group, and requires CDC to be re-enabled after DDL changes. See section 17.

4.4 Schema-level OneLake shortcut

The lakehouse exposes the PMS through **one shortcut at `Tables/pms`** pointing to the SQL database's `Tables/pms` folder. All four PMS tables (and any future table) appear in the lakehouse as `pms.Hotel`, `pms.RoomType`, `pms.Guest`, `pms.Reservation` – **zero copy**.

```
POST /v1/workspaces/{ws}/items/{lakehouseId}/shortcuts?shortcutConflictPolicy=CreateOrOverwrite
{
  "path": "Tables",
  "name": "pms",
  "target": { "oneLake": { "workspaceId": "{ws}", "itemId": "{sqlDatabaseId}", "path": "Tables/pms" } }
}
```

4.5 File landing (Salesforce, SAP, airlines)

Source file	Landing path	Bronze/silver table
salesforce_privilege_club_members.csv	`Files/landing/salesforce/privilege_club_members.csv`	`crm.privilege_club_member`
salesforce_guest_cases.csv	`Files/landing/salesforce/guest_cases.csv`	`crm.guest_case`
sap_hotel_costs_monthly.csv	`Files/landing/sap/hotel_costs_monthly.csv`	`erp.hotel_cost_monthly`
airport_seat_capacity_weekly.csv	`Files/landing/external/airport_seat_capacity_weekly.csv`	`ext.airport_seat_capacity`

Files are uploaded with the **OneLake DFS API** (create → append → flush, `x-ms-version: 2021-08-06`, storage audience token). In production Salesforce would use the Data Factory Salesforce connector or Copy job instead of hand-coded API extracts.

5. Lakehouse design and data lineage

`KyanosLakehouse` is created with `enableSchemas: true`. Schemas separate source-aligned data from serving data:

Schema	Purpose	Written by
pms	Mirrored PMS tables (shortcut, read-only)	Mirroring
crm	Salesforce Privilege Club and cases	01_Ingest
erp	SAP monthly costs	01_Ingest
ext	Airline seat capacity	01_Ingest
gold	Star schema for BI + forecast + ontology entities	02, 03, 04

5.1 Table inventory

Schema	Table	Rows	Cols	Role
crm	guest_case	17,379	11	Bronze/silver (typed landing)
crm	privilege_club_member	43,250	15	Bronze/silver (typed landing)
erp	hotel_cost_monthly	2,148	9	Bronze/silver (typed landing)
ext	airport_seat_capacity	1,884	8	Bronze/silver (typed landing)
gold	dim_airport	12	3	Gold dimension
gold	dim_date	1,095	13	Gold dimension
gold	dim_guest	255,848	12	Gold dimension
gold	dim_hotel	24	15	Gold dimension
gold	fact_airport_seats	1,872	6	Gold fact
gold	fact_guest_case	17,379	9	Gold fact
gold	fact_hotel_cost	2,148	5	Gold fact
gold	fact_hotel_day	26,280	9	Gold fact
gold	fact_occupancy_forecast	4,390	16	Gold fact
gold	fact_reservation	482,961	25	Gold fact
gold	iq_airport	12	5	Ontology entity
gold	iq_hotel	24	15	Ontology entity
gold	iq_open_case	124	9	Ontology entity
gold	iq_tour_operator	6	5	Ontology entity
gold	iq_upcoming_stay	3,600	11	Ontology entity
gold	iq_vip_guest	2,943	8	Ontology entity
pms	Guest	255,848	9	Shortcut → mirrored PMS
pms	Hotel	24	16	Shortcut → mirrored PMS
pms	Reservation	484,782	22	Shortcut → mirrored PMS
pms	RoomType	117	6	Shortcut → mirrored PMS

Row counts were taken from the lakehouse SQL analytics endpoint on 7 Oct 2026 (pms.Reservation includes live simulator bookings).

5.2 Data lineage

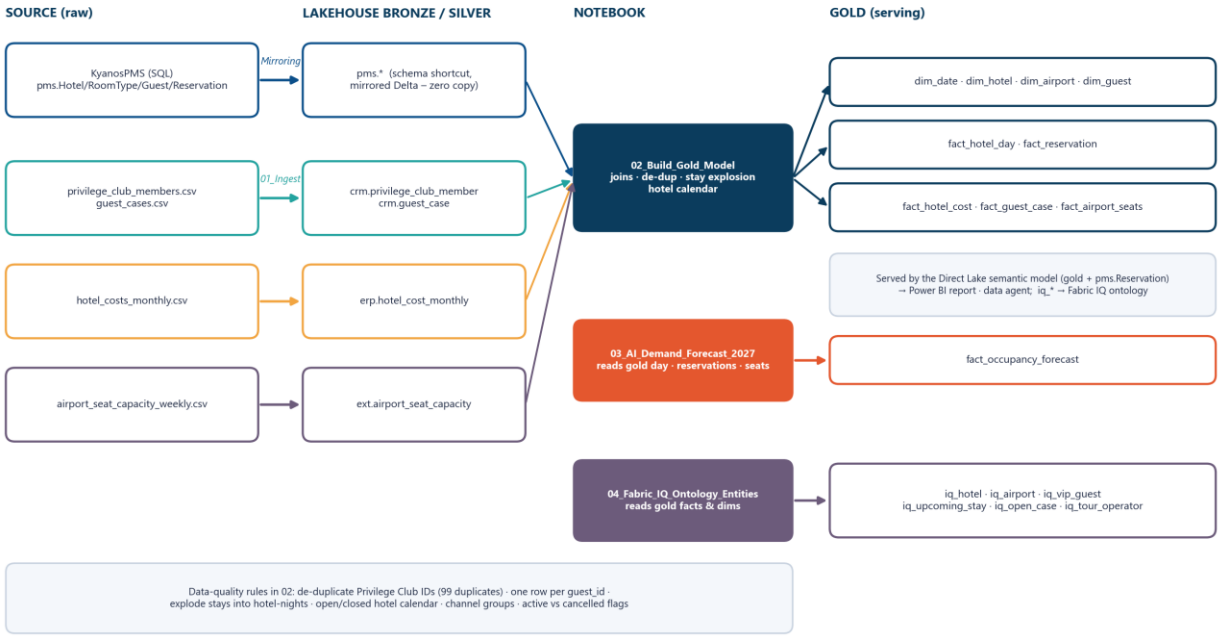


Figure 2 – Data lineage from source to gold

6. Notebooks and transformation logic

Four PySpark notebooks are deployed in Fabric git format (`notebook-content.py`) with the lakehouse attached as default, and executed with the **Jobs API** (`jobType=RunNotebook`). Typical duration on F8: 65–170 seconds.

Notebook	Inputs	Outputs	Key logic
01_Ingest_Salesforce_SAP_Airports	Files/landing/*.csv; pms.Reservation (check)	crm.*, erp.*, ext.*	CSV with schema inference + explicit casts; lineage columns _ingested_at, _source_file
02_Build_Gold_Model	pms.*, crm.*, erp.*, ext.*	gold.dim_* and gold.fact_* (10 tables)	Conformed dimensions, data-quality de-dup, stay explosion to hotel-nights, hotel calendar
03_AI_Demand_Forecast_2027	gold.fact_reservation, fact_hotel_day, dim_hotel, fact_airport_seats	gold.fact_occupancy_forecast; MLflow run + registered model	Booking-pace model, see section 11
04_Fabric_IQ_Ontology_Entities	gold dims and facts	gold.iq_* (6 tables)	Business entities for the ontology (VIP guests, upcoming stays, open cases …)

6.1 Gold table derivations

Gold table	Grain	Derivation
`dim_date`	Day 2025-01-01 → 2027-12-31	Generated sequence; year, quarter, month, week_start, weekday, season (Peak/High/Shoulder/Winter), is_actual (≤ snapshot), is_closed_ytd (month < October)
`dim_hotel`	Hotel	pms.Hotel renamed; short name; operating model; GM name and email (RLS)
`dim_airport`	Airport	12 airports with name and region
`dim_guest`	Guest	pms.Guest LEFT JOIN de-duplicated crm.privilege_club_member on loyalty number; tier, opt-in, preference; dropDuplicates(guest_id)
`fact_reservation`	Reservation	pms.Reservation + room type; lead_time_days; channel_group (Direct / OTA / Tour Operator & Wholesale / Corporate & MICE); is_cancelled, is_active
`fact_hotel_day`	Hotel × day	Active stays exploded into nights (revenue spread evenly) + arrivals, cross-joined with the hotel open/closed calendar → rooms_available, rooms_sold, revenue
`fact_hotel_cost`	Hotel × month × cost line	SAP plant code mapped to hotel_id; amount in EUR
`fact_guest_case`	Case	Salesforce case with hotel_id via hotel code; category, sub_category, CSAT
`fact_airport_seats`	Airport × week	Weekly arriving seats and flights (2025+)
`fact_occupancy_forecast`	Hotel × day (Jan–Sep 2027)	Notebook 03 output: OTB, LY, forecast rooms and revenue, drivers, model_run_id

6.2 Data-quality rules

- **Duplicate loyalty IDs:** 99 Privilege Club IDs are duplicated across Salesforce accounts. Without handling, the join produced duplicate `guest_id` values and broke the many-to-one relationship in the model. Notebook 02 keeps one member record per ID and one row per guest.
- **Stay explosion:** each active reservation becomes one row per night, so occupancy and RevPAR are exact at day level.
- **Hotel calendar:** `rooms_available` is zero when a seasonal hotel is closed, so occupancy is never diluted by closed days.
- **Status logic:** only Checked-Out, In-House and Confirmed stays count as active; cancellations and no-shows are excluded from occupancy.

7. Semantic model and table relationships

Property	Value
Name	Kyanos Hotels Model
Storage mode	Direct Lake on OneLake (expression DL_Lakehouse → AzureStorage.DataLake on the lakehouse)
Compatibility level	1702 · culture en-US · discourageImplicitMeasures
Definition format	TMDL, deployed with the Fabric items API (format TMDL)
Tables / relationships / measures	11 / 15 / 53
Date table	dim_date (dataCategory Time, date column isKey)
Refresh	None scheduled. Direct Lake reframes automatically on new Delta commits; a full refresh (reframe) is requested after each deployment
Security	RLS roles: Crete Region Manager, Hotel General Manager

```
expression DL_Lakehouse =
    let
        Source =
AzureStorage.DataLake("https://onelake.dfs.fabric.microsoft.com/<workspaceId>/<lakehouseId>",
[HierarchicalNavigation=true])
    in
        Source

partition fact_hotel_day = entity
    mode: directLake
    source
        entityName: fact_hotel_day
        schemaName: gold
        expressionSource: DL_Lakehouse
```

Table relationship diagram

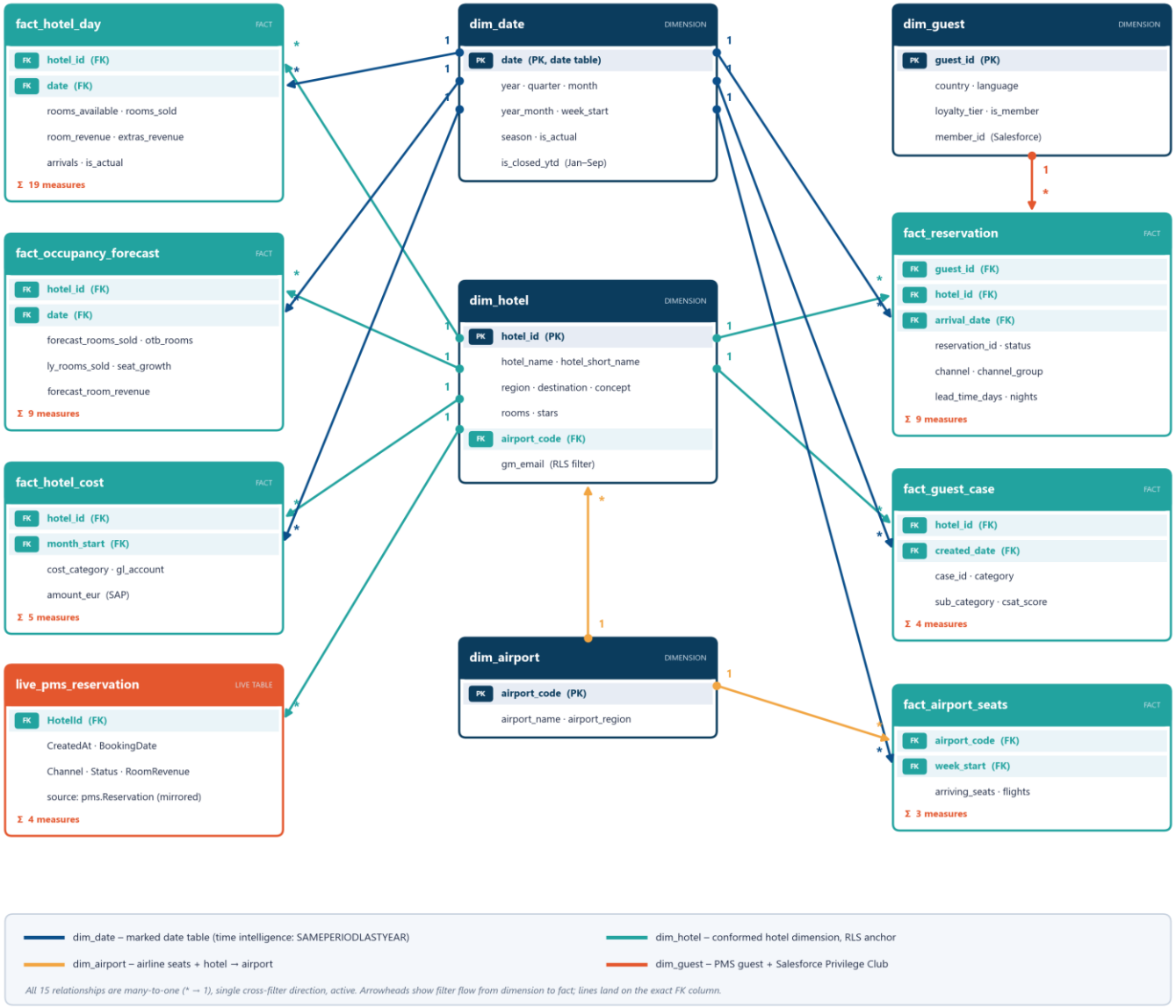


Figure 3 – Table relationship diagram of the semantic model (Direct Lake)

7.1 Relationships

From (many)	To (one)	Cardinality	Cross-filter	State
fact_hotel_day[hotel_id]	dim_hotel[hotel_id]	Many-to-one (*:1)	Single	Active
fact_hotel_day[date]	dim_date[date]	Many-to-one (*:1)	Single	Active
fact_reservation[hotel_id]	dim_hotel[hotel_id]	Many-to-one (*:1)	Single	Active
fact_reservation[arrival_date]	dim_date[date]	Many-to-one (*:1)	Single	Active
fact_reservation[guest_id]	dim_guest[guest_id]	Many-to-one (*:1)	Single	Active
fact_hotel_cost[hotel_id]	dim_hotel[hotel_id]	Many-to-one (*:1)	Single	Active
fact_hotel_cost[month_start]	dim_date[date]	Many-to-one (*:1)	Single	Active
fact_guest_case[hotel_id]	dim_hotel[hotel_id]	Many-to-one (*:1)	Single	Active
fact_guest_case[created_date]	dim_date[date]	Many-to-one (*:1)	Single	Active
fact_airport_seats[airport_code]	dim_airport[airport_code]	Many-to-one (*:1)	Single	Active
fact_airport_seats[week_start]	dim_date[date]	Many-to-one (*:1)	Single	Active
fact_occupancy_forecast[hotel_id]	dim_hotel[hotel_id]	Many-to-one (*:1)	Single	Active
fact_occupancy_forecast[date]	dim_date[date]	Many-to-one (*:1)	Single	Active
dim_hotel[airport_code]	dim_airport[airport_code]	Many-to-one (*:1)	Single	Active
live_pms_reservation[HotelId]	dim_hotel[hotel_id]	Many-to-one (*:1)	Single	Active

7.2 Relationship design notes

- **Star schema with conformed dimensions.** `dim_hotel` and `dim_date` are shared by all facts, so one slicer filters revenue, costs, cases, forecast and live data consistently.
- **Different date roles per fact:** stays by `arrival_date`, costs by `month_start`, cases by `created_date`, seats by `week_start`, hotel-day and forecast by `date`. Last-year measures therefore work for every fact.
- **Snowflake to airports:** `dim_hotel[airport_code]` → `dim_airport`. Because filters flow only from `dim_airport` to `dim_hotel`, airline-seat measures use `TREATAS(VALUES(dim_hotel[airport_code]), dim_airport[airport_code])` to respond to hotel filters without a bidirectional relationship.
- **Live table:** `live_pms_reservation` maps directly to the mirrored `pms.Reservation` (schema `pms`) and joins only to `dim_hotel` – it powers the Live PMS Feed page without any refresh.
- **All single-direction:** no bidirectional or many-to-many relationships, so there are no ambiguous paths and RLS on `dim_hotel` propagates to every fact.

7.3 Row-level security

Role	Table filter	Use
Crete Region Manager	`dim_hotel : [region] = "Crete"`	Regional manager sees the 6 Crete hotels
Hotel General Manager	`dim_hotel : [gm_email] = USERPRINCIPALNAME()`	Each GM sees only their own hotel (gm_email = signed-in user)

Role members are assigned in the Power BI service. Test with **Security** → **Test as role** on the semantic model.

7.4 Column conventions

- Keys and raw numeric columns on facts are **hidden**; users work with measures and dimension attributes.

- `dim_date[month]` sorts by `month_num`, `dim_date[weekday]` by `weekday_num`; `dim_hotel` latitude/longitude carry geographic data categories.
- Numeric key columns use `summarizeBy: none` to avoid implicit aggregation; `discourageImplicitMeasures` is enabled.

8. DAX measure catalog

All 53 measures are defined in TMDL with a description and display folder. They are grouped below by host table. Formats use EUR, percentages with one decimal, and signed percentage points for changes.

8.1 fact_hotel_day (19 measures)

Measure	DAX expression	Format	Description
Rooms Available	`SUM(fact_hotel_day[rooms_available])`	#,0	Saleable room nights (hotel open days x rooms).
Rooms Sold	`SUM(fact_hotel_day[rooms_sold])`	#,0	Occupied room nights (on-the-books for future dates).
Occupancy %	`DIVIDE([Rooms Sold], [Rooms Available])`	0.0%	Rooms sold / rooms available.
Room Revenue	`SUM(fact_hotel_day[room_revenue])`	€#,0	Accommodation revenue in EUR.
F&B and Other Revenue	`SUM(fact_hotel_day[extras_revenue])`	€#,0	Food & beverage, spa and other outlet revenue in EUR.
Total Revenue	`SUM(fact_hotel_day[total_revenue])`	€#,0	Room + F&B and other revenue in EUR.
ADR	`DIVIDE([Room Revenue], [Rooms Sold])`	€#,0.00	Average daily rate = room revenue / rooms sold.
RevPAR	`DIVIDE([Room Revenue], [Rooms Available])`	€#,0.00	Revenue per available room = room revenue / rooms available.
TRevPAR	`DIVIDE([Total Revenue], [Rooms Available])`	€#,0.00	Total revenue per available room.
Guest Arrivals	`SUM(fact_hotel_day[arrivals])`	#,0	Number of arriving reservations.
Occupancy % LY	`CALCULATE([Occupancy %], SAMEPERIODLASTYEAR(dim_date[date]))`	0.0%	Occupancy for the same period last year.
ADR LY	`CALCULATE([ADR], SAMEPERIODLASTYEAR(dim_date[date]))`	€#,0.00	ADR for the same period last year.
RevPAR LY	`CALCULATE([RevPAR], SAMEPERIODLASTYEAR(dim_date[date]))`	€#,0.00	RevPAR for the same period last year.
Room Revenue LY	`CALCULATE([Room Revenue], SAMEPERIODLASTYEAR(dim_date[date]))`	€#,0	Room revenue same period last year.
Total Revenue LY	`CALCULATE([Total Revenue], SAMEPERIODLASTYEAR(dim_date[date]))`	€#,0	Total revenue same period last year.
Occupancy vs LY (pts)	`IF(NOT ISBLANK([Occupancy % LY]), [Occupancy %] - [Occupancy % LY])`	+0.0%;-0.0%;0.0%	Change in occupancy vs last year, in percentage points.
ADR vs LY %	`DIVIDE([ADR] - [ADR LY], [ADR LY])`	+0.0%;-0.0%;0.0%	ADR growth vs last year.
RevPAR vs LY %	`DIVIDE([RevPAR] - [RevPAR LY], [RevPAR LY])`	+0.0%;-0.0%;0.0%	RevPAR growth vs last year.
Total Revenue vs LY %	`DIVIDE([Total Revenue] - [Total Revenue LY], [Total Revenue LY])`	+0.0%;-0.0%;0.0%	Total revenue growth vs last year.

8.2 fact_hotel_cost (5 measures)

Measure	DAX expression	Format	Description
Operating Costs	`SUM(fact_hotel_cost[amount_eur])`	€#,0	Operating costs from SAP in EUR.
GOP	`[Total Revenue] - [Operating Costs]`	€#,0	Gross operating profit = total revenue - operating costs.
GOP Margin %	`DIVIDE([GOP], [Total Revenue])`	0.0%	GOP as a share of total revenue.

Measure	DAX expression	Format	Description
GOPPAR	`DIVIDE([GOP], [Rooms Available])`	€#,0.00	Gross operating profit per available room.
GOP Margin % LY	`CALCULATE([GOP Margin %], SAMEPERIODLASTYEAR(dim_date[date]))`	0.0%	GOP margin same period last year.

8.3 fact_reservation (9 measures)

Measure	DAX expression	Format	Description
Bookings	`CALCULATE(COUNTROWS(fact_reservation), fact_reservation[is_active])`	#,0	Active reservations (not cancelled / no-show), by arrival date.
Cancellations	`CALCULATE(COUNTROWS(fact_reservation), fact_reservation[is_cancelled])`	#,0	Cancelled reservations.
Cancellation Rate %	`DIVIDE([Cancellations], [Cancellations] + [Bookings])`	0.0%	Cancelled / all reservations.
Avg Lead Time (days)	`CALCULATE(AVERAGE(fact_reservation[lead_time_days]), fact_reservation[is_active])`	0	Average days between booking and arrival.
Avg Length of Stay	`CALCULATE(AVERAGE(fact_reservation[nights]), fact_reservation[is_active])`	0.0	Average nights per stay.
Booked Room Revenue	`CALCULATE(SUM(fact_reservation[room_revenue]), fact_reservation[is_active])`	€#,0	Room revenue of active reservations.
Direct Share %	`DIVIDE(CALCULATE([Booked Room Revenue], fact_reservation[channel_group] = "Direct"), [Booked Room Revenue])`	0.0%	Share of room revenue booked direct (web + call centre).
Direct Share % LY	`CALCULATE([Direct Share %], SAMEPERIODLASTYEAR(dim_date[date]))`	0.0%	Direct share same period last year.
Privilege Club Share %	`DIVIDE(CALCULATE([Bookings], dim_guest[is_member] = TRUE()), [Bookings])`	0.0%	Share of stays by Privilege Club members.

8.4 fact_guest_case (4 measures)

Measure	DAX expression	Format	Description
Guest Cases	`COUNTROWS(fact_guest_case)`	#,0	Number of Salesforce guest cases.
Complaints	`CALCULATE([Guest Cases], fact_guest_case[category] = "Complaint")`	#,0	Guest complaints.
Avg Case CSAT	`AVERAGE(fact_guest_case[csat_score])`	0.00	Average case satisfaction score (1-5).
Complaints per 1,000 Room Nights	`DIVIDE([Complaints], [Rooms Sold]) * 1000`	0.00	Complaints normalised by occupied room nights.

8.5 fact_airport_seats (3 measures)

Measure	DAX expression	Format	Description
Airline Seats	`CALCULATE(SUM(fact_airport_seats[arriving_seats]), TREATAS(VALUES(dim_hotel[airport_code]), dim_airport[airport_code]))`	#,0	Scheduled arriving airline seats.
Airline Seats LY	`CALCULATE([Airline Seats], SAMEPERIODLASTYEAR(dim_date[date]))`	#,0	Arriving seats same period last year.
Airline Seat Growth %	`DIVIDE([Airline Seats] - [Airline Seats LY], [Airline Seats LY])`	+0.0%;-0.0%;0.0%	Change in scheduled arriving seats vs last year.

8.6 fact_occupancy_forecast (9 measures)

Measure	DAX expression	Format	Description
Forecast Occupancy %	`DIVIDE(SUM(fact_occupancy_forecast[forecast_rooms_sold]), SUM(fact_occupancy_forecast[rooms_available]))`	0.0%	AI forecast occupancy for 2027.
Last Year Occupancy %	`DIVIDE(SUM(fact_occupancy_forecast[ly_rooms_sold]), SUM(fact_occupancy_forecast[rooms_available]))`	0.0%	Actual occupancy of the same weekdays in 2026.
On-the-Books %	`DIVIDE(SUM(fact_occupancy_forecast[otb_rooms]), SUM(fact_occupancy_forecast[rooms_available]))`	0.0%	Occupancy already booked for 2027 as of 6 Oct 2026.
On-the-Books LY %	`DIVIDE(SUM(fact_occupancy_forecast[ly_otb_rooms]), SUM(fact_occupancy_forecast[rooms_available]))`	0.0%	Occupancy that was booked for 2026 at the same point last year.
Forecast vs LY (pts)	`[Forecast Occupancy %] - [Last Year Occupancy %]`	+0.0%;-0.0%;0.0%	Forecast occupancy change vs 2026, in points.
Booking Pace vs LY (pts)	`[On-the-Books %] - [On-the-Books LY %]`	+0.0%;-0.0%;0.0%	On-the-books ahead (+) or behind (-) last year's pace.
Forecast Room Revenue	`SUM(fact_occupancy_forecast[forecast_room_revenue])`	€#,0	Forecast 2027 room revenue (LY ADR +4%).
Forecast Room Revenue vs LY %	`DIVIDE([Forecast Room Revenue] - SUM(fact_occupancy_forecast[ly_room_revenue]), SUM(fact_occupancy_forecast[ly_room_revenue]))`	+0.0%;-0.0%;0.0%	Forecast room revenue growth vs 2026.
Seat Growth at Airport %	`DIVIDE(SUMX(fact_occupancy_forecast, fact_occupancy_forecast[seat_growth] * fact_occupancy_forecast[rooms_available]), SUM(fact_occupancy_forecast[rooms_available]))`	+0.0%;-0.0%;0.0%	Airline seat capacity change at the hotel's airport, Summer 2027 vs 2026.

8.7 live_pms_reservation (4 measures)

Measure	DAX expression	Format	Description
PMS Reservations (live)	`COUNTROWS(live_pms_reservation)`	#,0	All reservations currently in the PMS.
Bookings Last 24h (live)	`VAR cutoff = NOW() - 1 RETURN COALESCE(CALCULATE(COUNTROWS(live_pms_reservation), live_pms_reservation[CreatedAt] >= cutoff), 0)`	#,0	Reservations saved in the PMS in the last 24 hours (rolling).
Booked Revenue Last 24h (live)	`VAR cutoff = NOW() - 1 RETURN COALESCE(CALCULATE(SUM(live_pms_reservation[RoomRevenue]), live_pms_reservation[CreatedAt] >= cutoff), 0)`	€#,0	Room revenue of reservations saved in the last 24 hours (rolling).
Last PMS Booking	`FORMAT(MAX(live_pms_reservation[CreatedAt]), "dd mmm yyyy hh:nn:ss")`	text	Timestamp of the latest reservation in the PMS.

8.8 DAX patterns used

8.8.1 Time intelligence on a marked date table

```
RevPAR LY = CALCULATE ( [RevPAR], SAMEPERIODLASTYEAR ( dim_date[date] ) )
```

Because every fact relates to `dim_date`, the same pattern works for occupancy, ADR, revenue, GOP, direct share and airline seats. Pages filter `dim_date[is_closed_ytd] = TRUE` so both years compare January–September.

8.8.2 Blank-safe comparisons

```
RevPAR vs LY % = DIVIDE ( [RevPAR] - [RevPAR LY], [RevPAR LY] )
Occupancy vs LY (pts) = IF ( NOT ISBLANK ( [Occupancy % LY] ), [Occupancy %] - [Occupancy % LY] )
```

The first version used `DIVIDE([RevPAR], [RevPAR LY]) - 1`, which returned −100% when last year was blank (e.g. 2025 has no prior year). The rewritten form returns blank instead.

8.8.3 Virtual relationship with TREATAS

```
Airline Seats =
CALCULATE (
    SUM ( fact_airport_seats[arriving_seats] ),
    TREATAS ( VALUES ( dim_hotel[airport_code] ), dim_airport[airport_code] )
)
```

Pushes the hotel selection onto the airport dimension so seat growth can be shown next to each hotel, keeping the physical relationship single-direction.

8.8.4 Rolling 24-hour live window

```
Bookings Last 24h (live) =
VAR cutoff = NOW () - 1
RETURN
    COALESCE (
        CALCULATE ( COUNTROWS ( live_pms_reservation ), live_pms_reservation[CreatedAt] >= cutoff ),
        0
    )
```

Replaced an earlier `BookingDate = TODAY()` filter that left the Live page empty at the start of a new day. `NOW()` is evaluated in UTC in the service; `CreatedAt` is stored in UTC by the simulator.

8.8.5 Weighted ratios instead of averages

```
Forecast Occupancy % =
DIVIDE ( SUM ( fact_occupancy_forecast[forecast_rooms_sold] ), SUM (
fact_occupancy_forecast[rooms_available] ) )

Seat Growth at Airport % =
DIVIDE (
    SUMX ( fact_occupancy_forecast, fact_occupancy_forecast[seat_growth] *
fact_occupancy_forecast[rooms_available] ),
    SUM ( fact_occupancy_forecast[rooms_available] )
)
```

Ratios are always re-computed from additive components (rooms sold ÷ rooms available), never averaged, so totals are correct at any grain. Seat growth is weighted by room capacity.

9. DAX queries (validation and analysis)

The queries below were executed against the deployed model through the Power BI REST API (`POST /groups/{ws}/datasets/{id}/executeQueries`) to validate measures and relationships. Results are the actual values returned on 7 October 2026.

9.1 Season-to-date KPIs, 2026 vs 2025

Headline KPIs for closed months (Jan–Sep) per year; validates time intelligence and the GOP join to SAP costs.

```
EVALUATE
SUMMARIZECOLUMNS (
    dim_date[year],
    TREATAS ( { TRUE () }, dim_date[is_closed_ytd] ),
    TREATAS ( { 2025, 2026 }, dim_date[year] ),
    "Occupancy", [Occupancy %],
    "ADR", [ADR],
    "RevPAR", [RevPAR],
    "RevPAR vs LY", [RevPAR vs LY %],
    "Total Revenue", [Total Revenue],
    "GOP Margin", [GOP Margin %],
    "Direct Share", [Direct Share %]
)
ORDER BY dim_date[year]
```

year	Occupancy	ADR	RevPAR	RevPAR vs LY	Total Revenue	GOP Margin	Direct Share
2025	83.2%	€300.02	€249.56	(blank)	€362,577,449	39.5%	34.5%
2026	84.3%	€318.45	€268.46	7.6%	€384,982,838	40.2%	38.3%

9.2 Weakest hotels vs last year and airline seats

Bottom three hotels by RevPAR growth with the seat change at their airport (TREATAS bridge dim_hotel → dim_airport).

```
EVALUATE
TOPN (
    3,
    SUMMARIZECOLUMNS (
        dim_hotel[hotel_name],
        dim_hotel[airport_code],
        TREATAS ( { 2026 }, dim_date[year] ),
        TREATAS ( { TRUE () }, dim_date[is_closed_ytd] ),
        "RevPAR vs LY", [RevPAR vs LY %],
        "Occupancy vs LY (pts)", [Occupancy vs LY (pts)],
        "Airline Seat Growth", [Airline Seat Growth %]
    ),
    [RevPAR vs LY], ASC
)
ORDER BY [RevPAR vs LY] ASC
```

hotel_name	airport_code	RevPAR vs LY	Occupancy vs LY (pts)	Airline Seat Growth
Kyanos Kos Psalidi Suites	KGS	-8.1%	-6.8 pts	-12.9%
Kyanos Kos Marmari	KGS	-6.8%	-4.9 pts	-12.9%
Kyanos Zakynthos Laganas	ZTH	-6.2%	-4.1 pts	-10.2%

9.3 Guest cases at Kyanos Kos Marmari (2026)

Salesforce case mix for one hotel; surfaces the air-conditioning complaint cluster.

```
EVALUATE
TOPN (
    4,
    SUMMARIZECOLUMNS (
        fact_guest_case[sub_category],
        TREATAS ( { "Kyanos Kos Marmari" }, dim_hotel[hotel_name] ),
        TREATAS ( { 2026 }, dim_date[year] ),
        "Cases", [Guest Cases],
        "Avg CSAT", [Avg Case CSAT]
    ),
    [Cases], DESC
)
ORDER BY [Cases] DESC
```

sub_category	Cases	Avg CSAT
Air conditioning	157	2.46
Compliment - staff	39	4.74
Special request	38	4.13
Food & beverage	35	2.63

9.4 Summer 2027 AI forecast – top movers

Hotels with the largest forecast occupancy change vs 2026 and their drivers.

```
EVALUATE
TOPN (
    5,
    SUMMARIZECOLUMNS (
        dim_hotel[hotel_name],
        "Forecast Occ", [Forecast Occupancy %],
        "2026 Occ", [Last Year Occupancy %],
        "Change (pts)", [Forecast vs LY (pts)],
        "Seat Growth", [Seat Growth at Airport %],
        "Pace vs LY", [Booking Pace vs LY (pts)]
    ),
    [Change (pts)], DESC
)
ORDER BY [Change (pts)] DESC
```

hotel_name	Forecast Occ	2026 Occ	Change (pts)	Seat Growth	Pace vs LY
Kyanos Olympia Riviera	86.8%	82.4%	+4.4 pts	15.4%	+1.3 pts
Kyanos Chania Kalyves	88.9%	85.1%	+3.8 pts	12.1%	+1.0 pts
Kyanos Thessaloniki Center	81.2%	77.8%	+3.4 pts	8.9%	+0.1 pts
Kyanos Heraklion City	80.8%	77.8%	+3.0 pts	8.0%	-0.0 pts
Kyanos Rethymno Mare	89.3%	86.6%	+2.8 pts	12.2%	+1.2 pts

9.5 Group-level 2027 outlook

Single-row summary of the forecast page cards.

```
EVALUATE
ROW (
    "Forecast Occ 2027", [Forecast Occupancy %],
    "Actual Occ 2026", [Last Year Occupancy %],
    "On the books", [On-the-Books %],
    "Forecast Room Revenue", [Forecast Room Revenue],
    "vs 2026", [Forecast Room Revenue vs LY %]
)
```

Forecast Occ 2027	Actual Occ 2026	On the books	Forecast Room Revenue	vs 2026
86.0%	84.4%	7.1%	€319,509,787	5.4%

9.6 Privilege Club tier mix (2026)

Stays by Salesforce loyalty tier through dim_guest.

```
EVALUATE
SUMMARIZECOLUMNS (
    dim_guest[loyalty_tier],
    TREATAS ( { 2026 }, dim_date[year] ),
    TREATAS ( { TRUE () }, dim_date[is_closed_ytd] ),
    "Stays", [Bookings]
)
ORDER BY [Stays] DESC
```

loyalty_tier	Stays
Non-member	126,041
Silver	20,501
Gold	15,463
Blue	9,649
Platinum	5,797

9.7 GOP margin by hotel concept (2026 vs 2025)

Profitability by concept combining PMS revenue and SAP costs.

```
EVALUATE
SUMMARIZECOLUMNS (
    dim_hotel[concept],
    TREATAS ( { 2026 }, dim_date[year] ),
    TREATAS ( { TRUE () }, dim_date[is_closed_ytd] ),
    "GOP Margin", [GOP Margin %],
    "GOP Margin LY", [GOP Margin % LY],
    "GOPPAR", [GOPPAR]
)
ORDER BY [GOP Margin] DESC
```

concept	GOP Margin	GOP Margin LY	GOPPAR
Adults-Only Resort	56.8%	55.8%	€346.70
Luxury Resort	48.8%	48.0%	€224.00
Family Resort	35.1%	33.8%	€107.53
All-Inclusive Resort	24.5%	25.4%	€59.33
City Hotel	23.3%	22.6%	€47.90

9.8 Live PMS – last 24 hours by hotel

Rolling 24-hour window over the mirrored PMS table (Direct Lake, no refresh needed).

```
EVALUATE
TOPN (
    5,
    SUMMARIZECOLUMNS (
        dim_hotel[hotel_short_name],
        "Bookings 24h", [Bookings Last 24h (live)],
        "Revenue 24h", [Booked Revenue Last 24h (live)]
    ),
    [Bookings 24h], DESC
)
ORDER BY [Bookings 24h] DESC
```

hotel_short_name	Bookings 24h	Revenue 24h
Athens Acropolis	94	€52,671
Heraklion City	64	€19,719
Thessaloniki Center	54	€14,224
Malia Riviera	31	€52,346
Olympia Riviera	21	€68,619
Faliraki Family	21	€35,156

10. Power BI report

Kyanos Hotels - Commercial Performance is deployed in **PBIR** format (definition.pbir, report.json, pages/, visuals/) bound to the semantic model by ID, with a custom theme registered as a static resource.

10.1 Design system

Element	Specification
Canvas	1280 × 720, FitToPage, page background #EEF3F8
Theme	Custom "Kyanos Aegean" – data colours #0B4F8A, #22A39F, #F2A541, #E4572E, #7A9E7E; Segoe UI; navy #0B3C5D callouts
Header band	Full-width navy textbox: brand line, page title (18pt), context subtitle (sources, period)
KPI tiles	White rounded container (textbox) + transparent card for the value + transparent card for the comparison (vs LY)
Visual containers	White background, 10px radius, light border, 11pt bold titles, optional subtitles carrying the insight
Filters	Page-level filters on dim_date (year, is_closed_ytd) so every visual compares the same months

10.2 Pages

10.2.1 Executive Overview

Page filters: `dim_date[year] = 2026L`, `dim_date[is_closed_ytd] = true`

Visual	Title	Fields / measures
card ×6	KPI tiles	Total revenue (vs last year); Occupancy (pts vs last year); ADR (vs last year); RevPAR (vs last year); GOP margin (last year); Direct booking share (last year)
lineChart	Total revenue by month — 2026 vs 2025	month, 2026, 2025
treemap	Revenue mix by destination	Destination, Total Revenue
clusteredColumnChart	GOP margin by hotel concept (SAP)	concept, GOP Margin %, GOP Margin % LY
barChart	RevPAR growth vs last year, by hotel	Hotel, RevPAR vs LY

10.2.2 Hotel Scorecard

Page filters: `dim_date[year] = 2026L`, `dim_date[is_closed_ytd] = true`

Visual	Title	Fields / measures
tableEx	Hotel KPIs vs last year	Hotel, Region, Concept, Occupancy, Occ vs LY, ADR, ADR vs LY, RevPAR, RevPAR vs LY, GOP %, GOPPAR, Case CSAT, Complaints /1k RN
scatterChart	Rate vs volume — ADR by occupancy (bubble = revenue)	Hotel, Occupancy %, ADR, Total Revenue, concept
barChart	Salesforce guest cases by type	Case type, Guest Cases, category

10.2.3 Channels & Guests

Page filters: `dim_date[year] = 2026L`, `dim_date[is_closed_ytd] = true`

Visual	Title	Fields / measures
card ×6	KPI tiles	Direct booking share (last year); Privilege Club stays; Avg lead time (days); Avg length of stay; Cancellation rate; Stays

Visual	Title	Fields / measures
lineChart	Direct booking share by month — the direct strategy is working	month, 2026, 2025
barChart	Room revenue by booking channel	Channel, Room revenue
donutChart	Stays by Privilege Club tier (Salesforce)	Tier, Stays
clusteredColumnChart	Average booking lead time (days)	Channel group, Lead time
barChart	Source markets	Country, Stays

10.2.4 2027 Demand Outlook (AI)

Page filters: `dim_date[year] = 2027L`, `dim_date[is_closed_ytd] = true`

Visual	Title	Fields / measures
card ×5	KPI tiles	Forecast occupancy 2027 (pts vs 2026); 2026 actual occupancy; Already on the books (pace vs LY (pts)); Forecast room revenue (vs 2026); Airline seats vs 2026
lineChart	Occupancy by month — forecast vs 2026 actual vs booked so far	month, Forecast 2027, Actual 2026, On the books today
clusteredColumnChart	Published airline seat capacity, Summer 2027 vs 2026	Airport, Seat growth
barChart	Forecast occupancy change vs 2026, by hotel	Hotel, Forecast vs 2026 (pts)
tableEx	Drivers per hotel	Hotel, Airport, Seats, Pace, 2026, 2027 fcst, Fcst room rev

10.2.5 Live PMS Feed

Visual	Title	Fields / measures
card ×4	KPI tiles	Bookings in the last 24 hours; Room revenue booked, last 24 hours; Reservations in the PMS; Latest booking saved at (UTC)
tableEx	Latest reservations in the PMS	Saved at (UTC), Confirmation, Hotel, Channel, Arrival, Nights, Board, Status
barChart	Bookings in the last 24 hours, by hotel	Hotel, Bookings, last 24h
donutChart	Last 24 hours bookings by channel	Channel, Bookings, last 24h

10.3 Screenshots

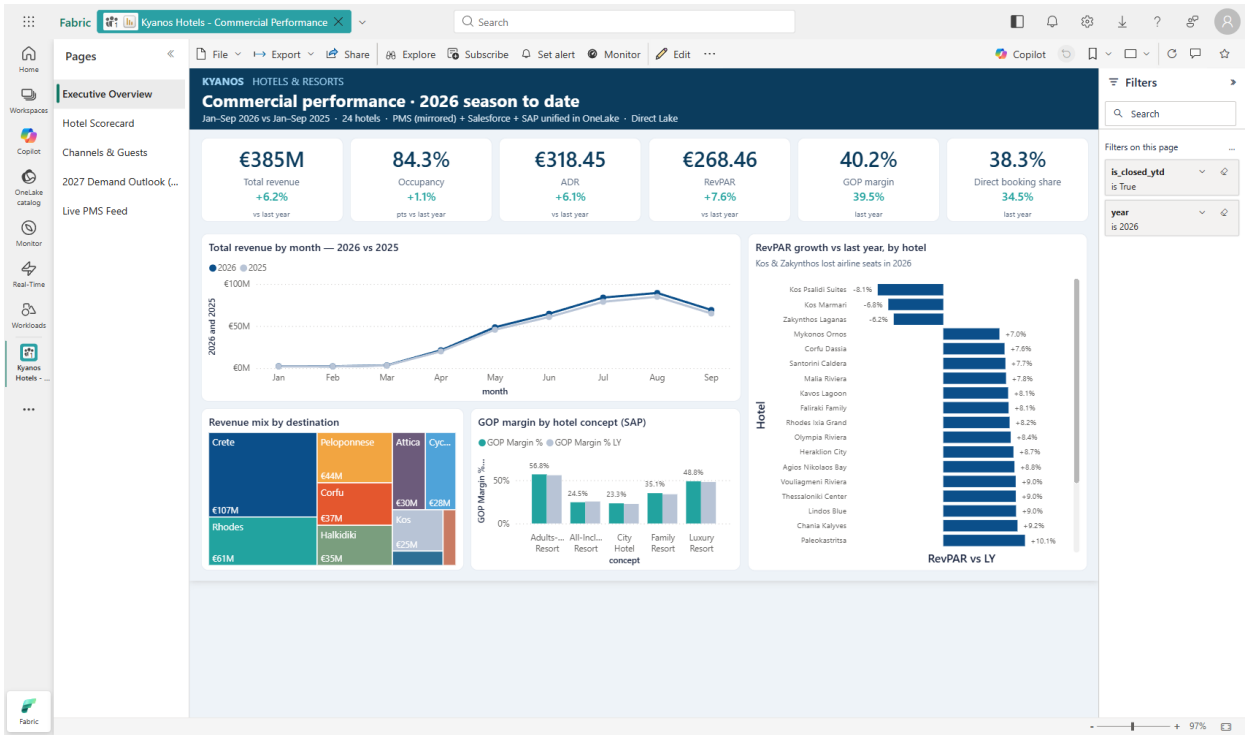


Figure 4 – Executive Overview

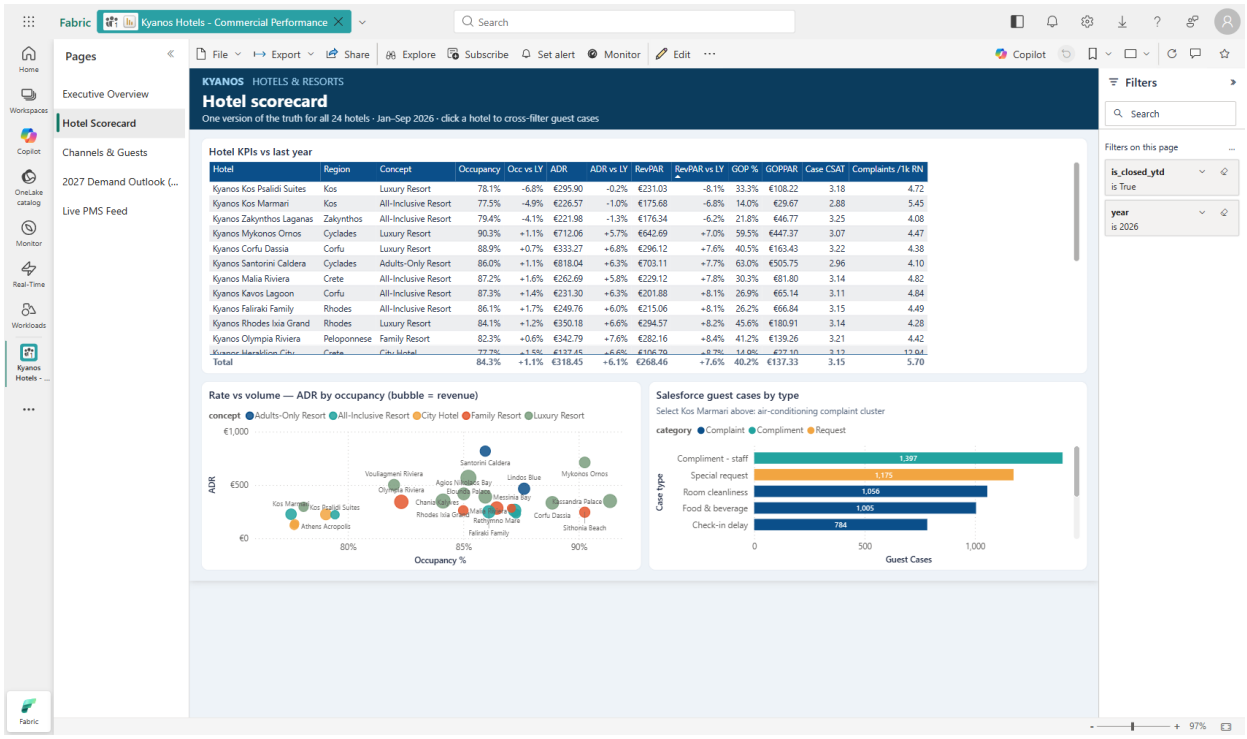


Figure 5 – Hotel Scorecard

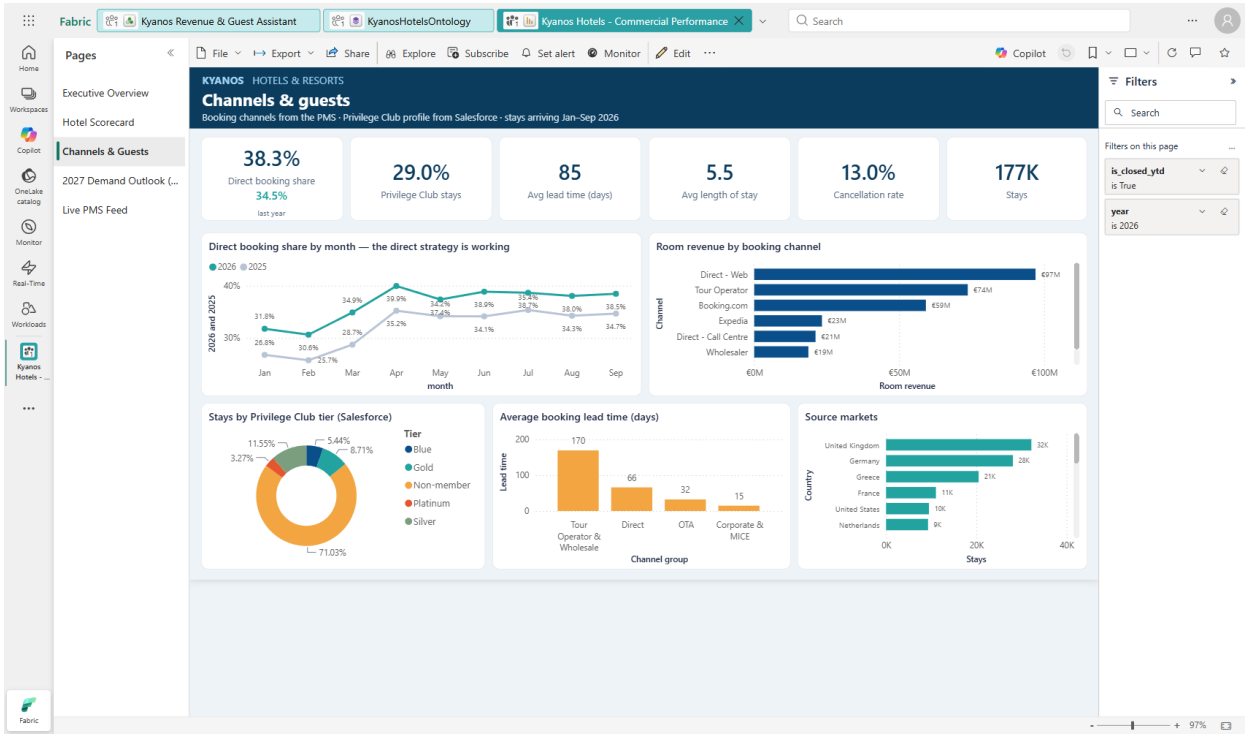


Figure 6 – Channels & Guests

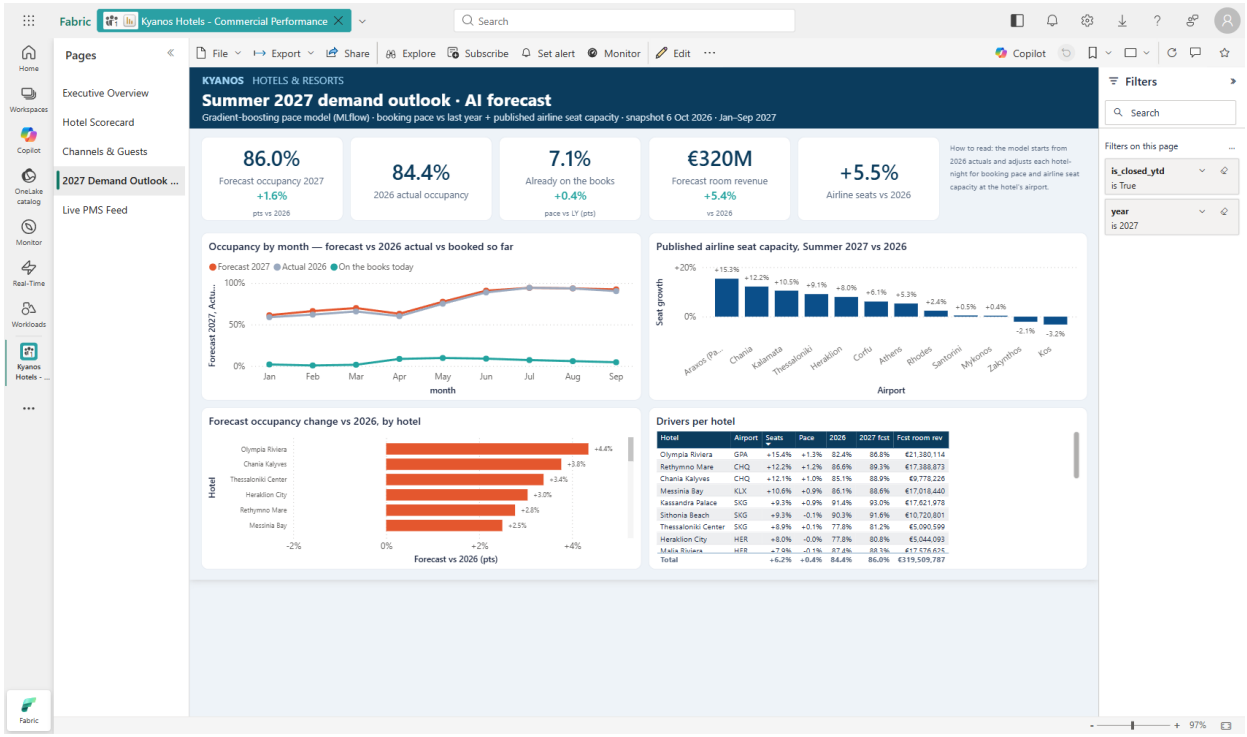


Figure 7 – 2027 Demand Outlook (AI)

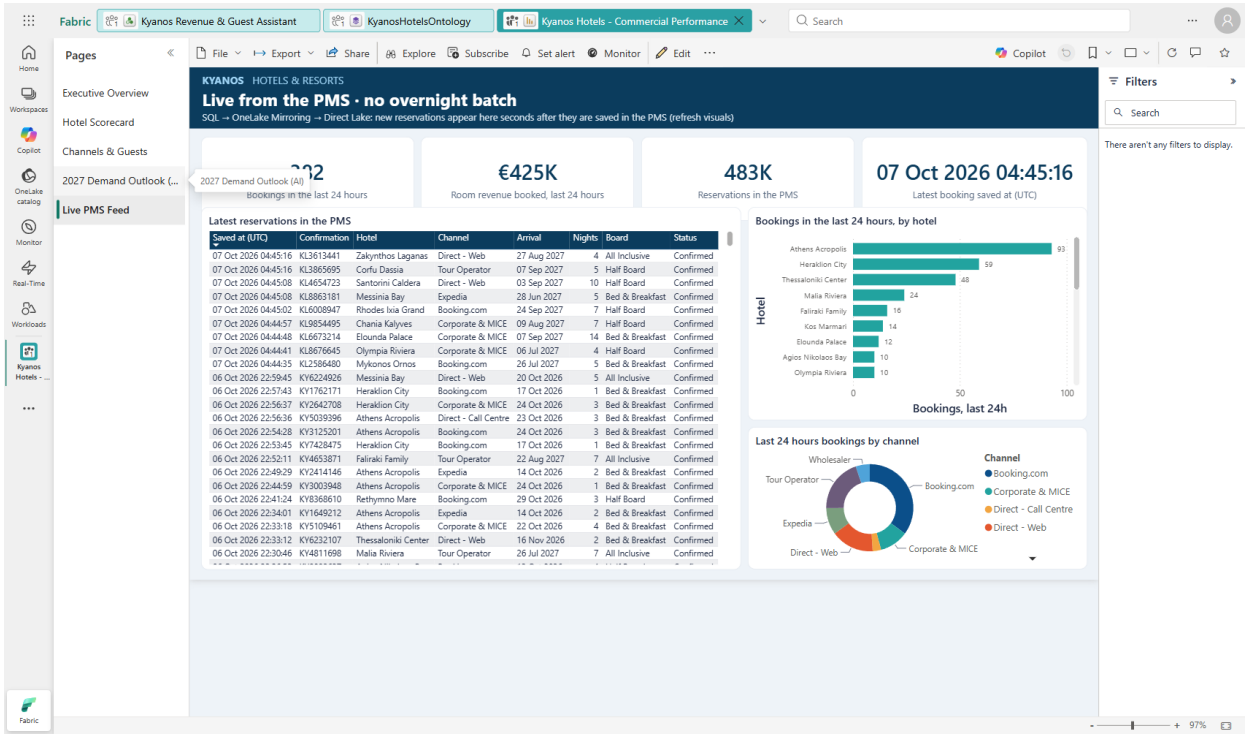


Figure 8 – Live PMS Feed

11. AI demand forecast

11.1 Business question

"How full will each hotel be next season, given what is already on the books and what the airlines have scheduled?" Revenue managers answer this every October when contracting tour operators and setting rates for the next summer.

11.2 Approach

- **Target:** change in daily occupancy versus the same weekday last year (`delta_vs_ly = occ − ly_occ`). Predicting the change, not the level, stops the model from simply memorising each hotel.
- **Training data:** every open hotel-day of 2026 up to the snapshot (4,534 rows) with its final occupancy, last year's occupancy and the drivers as they stood on 6 Oct 2025.
- **Scoring data:** every open hotel-day of Jan–Sep 2027 (4,390 rows) with drivers as of the 6 Oct 2026 snapshot.

Feature	Definition	Constraint
`seat_growth_28d`	Scheduled arriving seats at the hotel's airport vs the same week last year, 28-day centred rolling mean	Monotonic increasing
`pace_vs_ly_28d`	(rooms on the books − rooms on the books at the same point last year) ÷ rooms available, 28-day rolling mean	Monotonic increasing
`headroom`	1 − last-year occupancy (how much room there is to grow)	None

- **Model:** scikit-learn HistGradientBoostingRegressor (max_iter 300, learning_rate 0.04, max_leaf_nodes 15, min_samples_leaf 60, l2 1) with monotonic constraints so that more seats or a stronger pace can only raise the forecast.
- **Forecast:** `forecast_occ = clip(ly_occ + predicted change, otb_occ, 99.5%)` – never below what is already booked. Forecast room revenue = forecast rooms × last-year ADR × 1.04.

11.3 Accuracy and tracking

Metric	Value
Holdout MAE (20% random hotel-days)	3.31 occupancy points
Baseline: 'same as last year'	4.09 occupancy points
Improvement	19% lower error than the baseline
MLflow experiment	kyanos-occupancy-forecast (run pace-model-summer-2027; params, metrics, model, input example)
Registered model	kyanos-occupancy-forecast (Fabric ML model item, versioned)

11.4 Results by hotel (Jan–Sep 2027)

Hotel	Airport	2026 occ.	2027 fcst	Change (pts)	Seat growth	Pace vs LY (pts)
Kyanos Olympia Riviera	GPA	82.4%	86.8%	+4.4	+15.4%	+1.3
Kyanos Chania Kalyves	CHQ	85.1%	88.9%	+3.8	+12.1%	+1.0
Kyanos Thessaloniki Center	SKG	77.8%	81.2%	+3.4	+8.9%	+0.1
Kyanos Heraklion City	HER	77.8%	80.9%	+3.1	+8.0%	+0.0

Hotel	Airport	2026 occ.	2027 fcst	Change (pts)	Seat growth	Pace vs LY (pts)
Kyanos Rethymno Mare	CHQ	86.6%	89.3%	+2.7	+12.2%	+1.2
Kyanos Messinia Bay	KLX	86.1%	88.6%	+2.5	+10.6%	+0.9
Kyanos Athens Acropolis	ATH	79.2%	81.6%	+2.4	+5.3%	+0.0
Kyanos Kavos Lagoon	CFU	87.3%	89.2%	+1.9	+6.4%	-0.1
Kyanos Paleokastritsa	CFU	87.2%	89.1%	+1.9	+6.3%	+0.9
Kyanos Agios Nikolaos Bay	HER	85.1%	87.0%	+1.9	+7.7%	+1.3
Kyanos Kassandra Palace	SKG	91.4%	93.0%	+1.6	+9.3%	+0.9
Kyanos Elounda Palace	HER	85.4%	86.8%	+1.4	+7.8%	+0.6
Kyanos Sithonia Beach	SKG	90.3%	91.6%	+1.3	+9.3%	-0.1
Kyanos Corfu Dassia	CFU	89.0%	90.0%	+1.0	+6.3%	+0.3
Kyanos Malia Riviera	HER	87.4%	88.3%	+0.9	+7.9%	-0.1
Kyanos Vouliagmeni Riviera	ATH	82.2%	82.9%	+0.7	+5.3%	+1.2
Kyanos Kos Marmari	KGS	77.6%	77.9%	+0.3	-3.0%	-0.6
Kyanos Kos Psalidi Suites	KGS	78.2%	78.5%	+0.3	-3.0%	-0.4
Kyanos Faliraki Family	RHO	86.3%	86.6%	+0.3	+2.0%	+-0.0
Kyanos Zakynthos Laganas	ZTH	79.5%	79.8%	+0.3	-2.2%	+0.0
Kyanos Rhodes Ixia Grand	RHO	84.2%	84.2%	+0.0	+2.2%	+0.0
Kyanos Lindos Blue	RHO	87.8%	87.7%	-0.1	+2.0%	+1.0
Kyanos Santorini Caldera	JTR	86.1%	85.4%	-0.7	+0.3%	+1.1
Kyanos Mykonos Ornos	JMK	90.4%	88.9%	-1.5	+0.1%	+1.5

Reading: Olympia Riviera (Araxos +15% seats) and Chania (+12%) lead; Kos and Zakynthos stay flat because airlines are still reducing capacity. City hotels gain on Athens/Heraklion/Thessaloniki capacity growth.

11.5 First attempt and lesson learned

The first model predicted occupancy directly from on-the-books, last-year occupancy, seats, hotel attributes and calendar. It reproduced last year almost exactly and even forecast growth for Kos despite seat cuts – it had learned hotel identity. Re-framing the target as the change versus last year, smoothing the drivers and adding monotonic constraints produced an explainable model that beats the naive baseline.

12. Fabric data agent

Property	Value
Name	Kyanos Revenue & Guest Assistant
Data source	Kyanos Hotels Model (type semantic_model) – all 11 tables, visible columns and all 53 measures selected
Definition	Files/Config/data_agent.json, draft/ and published/ stage_config.json + semantic-model datasource.json, publish_info.json
Languages	Greek and English (answers in the language of the question)
Typical response time	60–70 seconds (Standard runtime)
Channels	Fabric portal; publishable to Microsoft Teams and Microsoft 365 Copilot

12.1 Agent instructions

```
## Role
You are the **Kyanos Revenue & Guest Assistant** for Kyanos Hotels & Resorts (fictional Greek hotel group: 24 hotels - 20 seasonal resorts on Crete, Rhodes, Kos, Corfu, the Cyclades, Halkidiki, the Peloponnese, Zakynthos and Attica, plus 3 year-round city hotels in Athens, Heraklion and Thessaloniki).
Users are general managers, revenue managers, sales and the executive team. Answer in the language of the question (Greek or English).


## Time context
- Today / data snapshot = **6 October 2026**. Dates up to 6 Oct 2026 are actuals; later dates are on-the-books (future reservations) or forecast.
- "This season", "this year", "YTD" = **Jan-Sep 2026** (closed months: filter dim_date[is_closed_ytd] = TRUE and dim_date[year] = 2026). "Last year" = the same months of 2025.
- "Next season", "2027", "forecast", "outlook" = table fact_occupancy_forecast (Jan-Sep 2027), measures in folder "Forecast 2027".
- "Today", "live", "right now" = table live_pms_reservation (mirrored PMS, near real time); "bookings today" / "last 24 hours" = [Bookings Last 24h (live)] and [Booked Revenue Last 24h (live)] (rolling 24-hour window).


## KPI definitions - always use the model measures, never recompute
- Occupancy = [Occupancy %]; ADR = [ADR]; RevPAR = [RevPAR]; TRevPAR = [TRevPAR]; revenue = [Total Revenue] (rooms + F&B and other); room revenue = [Room Revenue].
- Versus last year: [RevPAR vs LY %], [ADR vs LY %], [Occupancy vs LY (pts)], [Total Revenue vs LY %].
- Profitability: [GOP], [GOP Margin %], [GOPPAR] (costs come from SAP, monthly).
- Channels: [Direct Share %] (direct web + call centre share of room revenue), channel / channel_group / tour_operator in fact_reservation. "OTA" = Booking.com + Expedia.
- Guests: dim_guest (country, loyalty_tier = Blue/Silver/Gold/Platinum/Non-member from Salesforce Privilege Club), [Privilege Club Share %].
- Service quality: fact_guest_case from Salesforce: [Complaints], [Avg Case CSAT] (1-5), [Complaints per 1,000 Room Nights], sub_category (e.g. "Air conditioning").
- Market: [Airline Seat Growth %] = scheduled airline seats at the hotel's airport vs last year.
- Forecast: [Forecast Occupancy %], [Last Year Occupancy %], [Forecast vs LY (pts)], [Booking Pace vs LY (pts)], [Seat Growth at Airport %], [Forecast Room Revenue].


## Business vocabulary
- Hotels are named "Kyanos <place>" (e.g. "Kyanos Elounda Palace"); users may say only the place ("Elounda", "Marmari", "Ixia"). Match on dim_hotel[hotel_name] or hotel_short_name.
- Region/destination values: Crete, Rhodes, Kos, Corfu, Cyclades, Halkidiki, Macedonia, Peloponnese, Attica, Zakynthos.
- Concepts: Luxury Resort, Adults-Only Resort, Family Resort, All-Inclusive Resort, City Hotel.
- "Underperforming" = lowest [RevPAR vs LY %]. "Pace" = [Booking Pace vs LY (pts)]. "VIP" = loyalty_tier Gold or Platinum.


## Answer style
Lead with the number(s), then one short line of interpretation (why, using drivers such as airline seats, channel mix, complaints). Use compact tables for rankings (max 10 rows). Format money in EUR, percentages with one decimal. If a question cannot be answered from the model, say so plainly and name the closest available data. Never invent hotels, guests or numbers.
```

12.2 Tested questions

Question	Answer (summary)
Which 3 hotels had the weakest RevPAR vs last year this season, and what happened to airline seats at their airports?	Kos Psalidi Suites −8.1%, Kos Marmari −6.8%, Zakynthos Laganas −6.2%; seats −12.9% (KGS) and −10.2% (ZTH). Interpretation: occupancy-driven decline caused by reduced air capacity rather than price. (≈66 s)

Question	Answer (summary)
Ποια ξενοδοχεία αναμένεται να έχουν τη μεγαλύτερη αύξηση πληρότητας το 2027 και γιατί;	Olympia Riviera +4.4, Chania Kalyves +3.8, Thessaloniki Center +3.4, Heraklion City +3.0, Rethymno Mare +2.8 pts – explained by double-digit growth in available airline seats, with booking pace a secondary factor. Answered in Greek. (≈70 s)

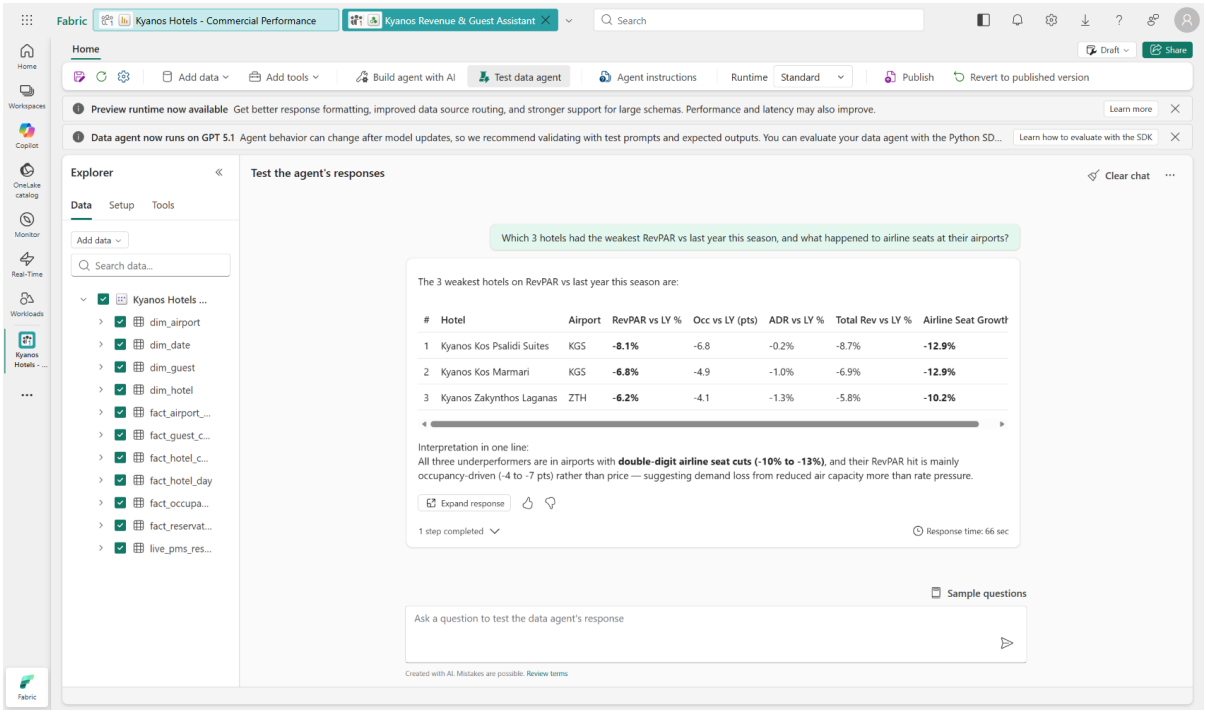


Figure 9 – Data agent answer (English)

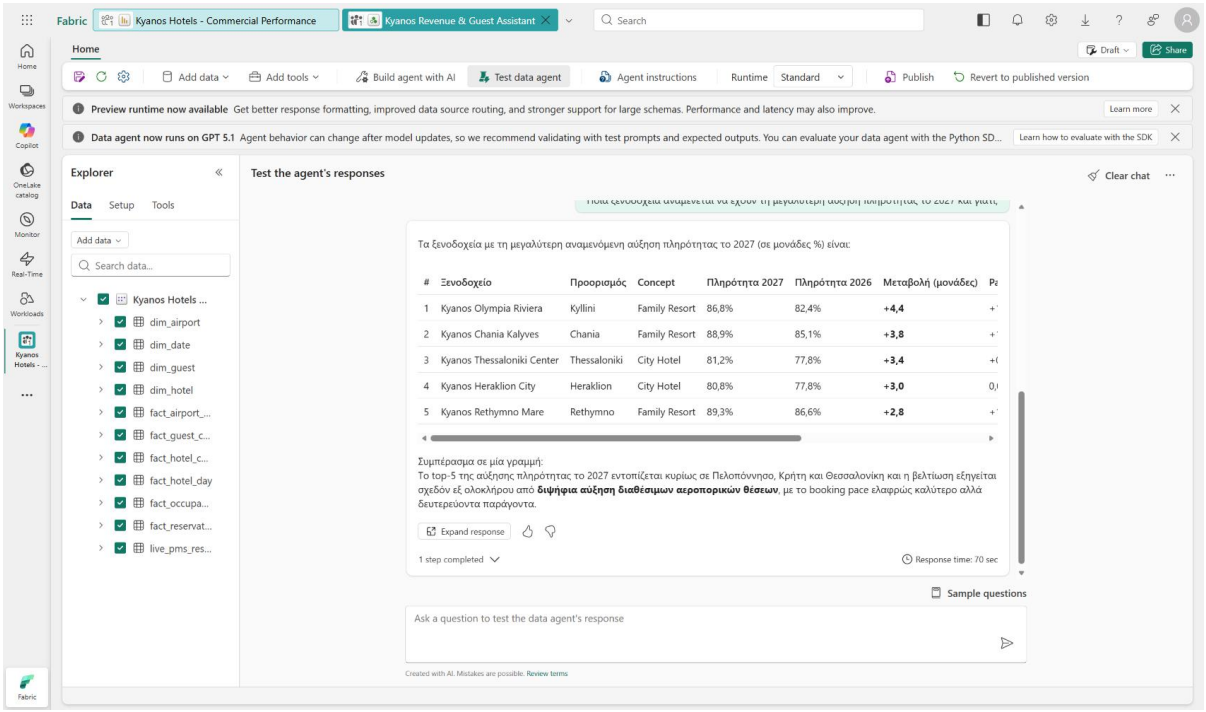


Figure 10 – Data agent answer (Greek)

13. Fabric IQ ontology

`KyanosHotelsOntology` turns gold tables into connected business concepts that people and agents can navigate (e.g. "which Platinum guests arrive at a hotel with open air-conditioning cases?"). It is deployed through the items API with entity types, data bindings to lakehouse tables (schema `gold`) and relationship contextualizations. Fabric automatically creates a companion lakehouse and graph model.

13.1 Entity types

Entity	Bound table	Key	Display property	Properties
Hotel	`gold.iq_hotel`	`hotel_id`	`hotel_name`	15
Airport	`gold.iq_airport`	`airport_code`	`airport_name`	5
VipGuest	`gold.iq_vip_guest`	`guest_id`	`guest_name`	8
UpcomingStay	`gold.iq_upcoming_stay`	`reservation_id`	`confirmation_no`	11
OpenCase	`gold.iq_open_case`	`case_id`	`case_id`	9
TourOperator	`gold.iq_tour_operator`	`tour_operator_id`	`tour_operator_name`	5

13.2 Relationships

Relationship	Source → target	Binding table	Join
served_by	Hotel → Airport	`gold.iq_hotel`	`hotel_id → airport_code`
stays_at	UpcomingStay → Hotel	`gold.iq_upcoming_stay`	`reservation_id → hotel_id`
booked_by	UpcomingStay → VipGuest	`gold.iq_upcoming_stay`	`reservation_id → guest_id`
sold_by	UpcomingStay → TourOperator	`gold.iq_upcoming_stay`	`reservation_id → tour_operator_id`
case_at	OpenCase → Hotel	`gold.iq_open_case`	`case_id → hotel_id`

13.3 Entity tables (notebook 04)

- `iq_hotel` – hotel attributes plus 2026 occupancy, RevPAR, RevPAR vs LY, average case CSAT and open cases.
- `iq_airport` – seat growth 2026 and 2027 per airport.
- `iq_vip_guest` – 2,943 Gold/Platinum members with upcoming stays; `iq_upcoming_stay` – 3,600 future stays of VIP guests.
- `iq_open_case` – 124 open Salesforce cases; `iq_tour_operator` – 6 tour operators with 2026 room nights and 2027 on-the-books.

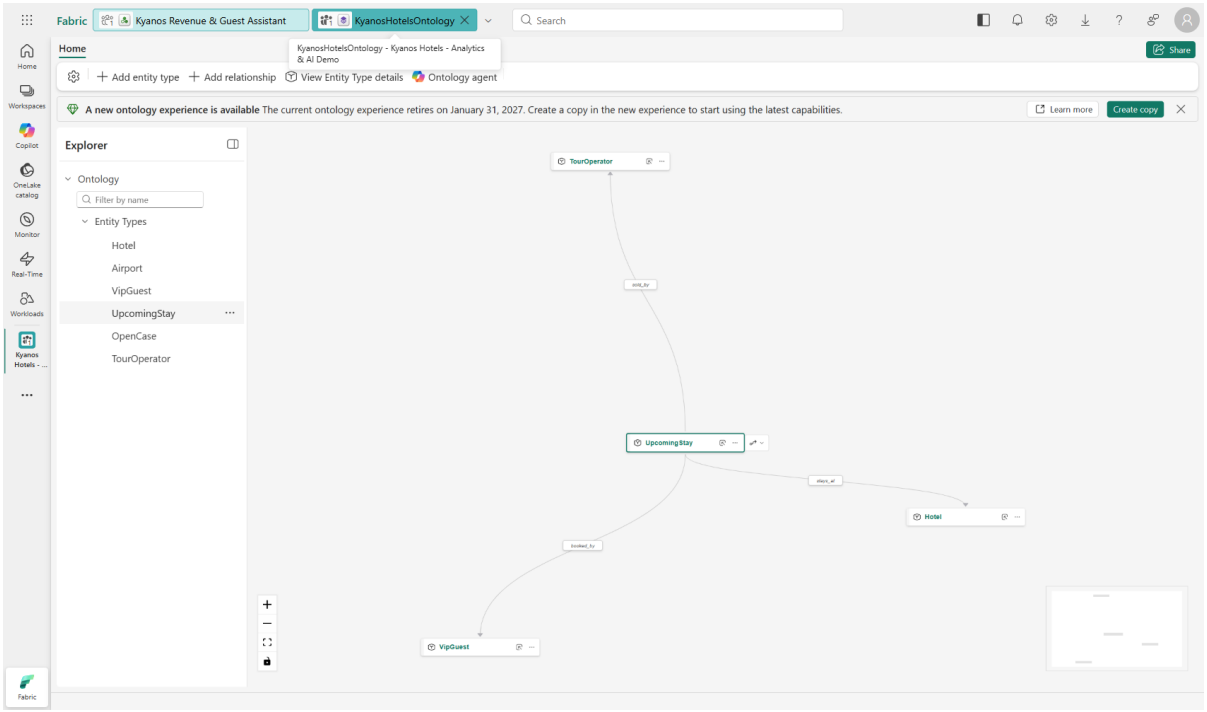


Figure 11 – Ontology graph around UpcomingStay

> **Note**
>
> The ontology editor shows a banner that a new ontology experience is available (the current one retires on 31 January 2027). The built-in **Ontology agent** (preview) did not respond to an automated test prompt; treat it as optional in demos.

14. Live PMS simulator and demo operations

14.1 Simulator

- `src/pms_live_sim.py` connects to KyanosPMS with an Entra token and inserts realistic reservations (1–3 every ~6 s) with confirmation numbers starting `KL`.
- Bookings respect hotel seasons (resorts get Summer 2027 arrivals, city hotels arrive within 75 days), channel mix, board types and rates derived from each hotel's recent Booking.com ADR.
- `burst N HOTEL` inserts a tour-operator group block (N rooms × 7 nights) – the visible "live moment" in the demo.
- The connection is re-established every 40 minutes and on errors, because Entra access tokens expire.

14.2 Control script

Command	Effect
`./demo-control.ps1 warmup`	Reframe the semantic model, sync the lakehouse SQL endpoint metadata, run warm-up DAX queries
`./demo-control.ps1 start`	Start the simulator in the background (PID in .sim.pid, log in sim.log)
`./demo-control.ps1 burst 25 KYN-OLY`	Insert a 25-room group block at Kyanos Olympia Riviera
`./demo-control.ps1 status`	Process state and number/value of simulator bookings
`./demo-control.ps1 stop | reset`	Stop the simulator | stop and delete all simulator bookings

14.3 Measured latency

Path	Observed
INSERT in KyanosPMS → new Delta version in OneLake	Seconds (new _delta_log entry immediately after insert)
INSERT → visible in Direct Lake model (DAX)	≈13 seconds for a 12-row burst
INSERT → visible in lakehouse SQL endpoint via shortcut	Not until a metadata sync (lazy for shortcut tables); refreshMetadata API forces it

14.4 Demo run sheet (summary)

1. 10:30 – `reset`, `warmup`, `start`; open workspace, report, data agent, SQL query editor, notebook 03, ontology.
2. Mirroring story → `burst` → Live PMS Feed shows the bookings in ~15 s.
3. Lakehouse schemas and shortcut → notebook 02 (data quality, Copilot) → Direct Lake model and RLS.
4. Report: Executive Overview (Kos/Zakynthos), Hotel Scorecard (A/C cluster), Channels (direct strategy), Copilot prompts.
5. 2027 AI outlook and MLflow → data agent in Greek → ontology → phased plan and next steps.

Full talk track: `docs/DEMO_SCRIPT.md` in the repository.

15. Build and deployment process

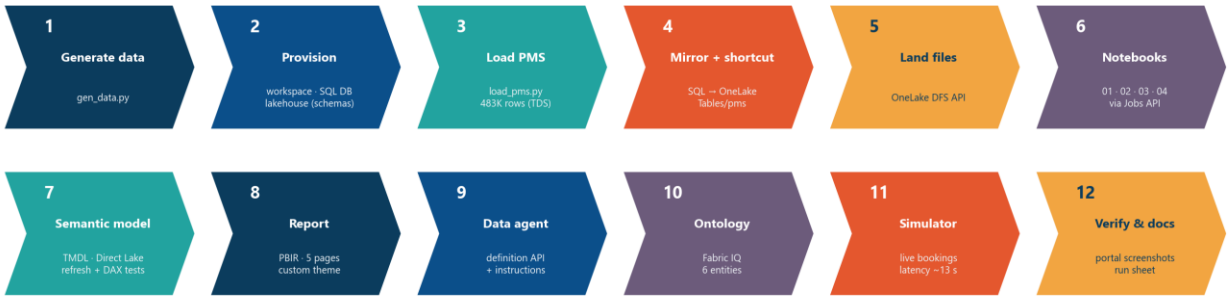


Figure 12 – Build process

#	Step	Script / API	What happens
1	Generate data	python src/gen_data.py	Synthetic PMS, Salesforce, SAP and airline datasets (seeded) into data/
2	Provision	deploy.py infra · POST /workspaces, /items	Workspace on capacity; Lakehouse (enableSchemas); SQL database; connection details captured in .state.json
3	Load PMS	python src/load_pms.py · TDS + Entra token	DDL for pms schema; 483K reservations loaded
4	Mirror + shortcut	deploy.py shortcut · POST .../shortcuts	Automatic SQL → OneLake replication; Tables/pms schema shortcut
5	Land files	deploy.py landing · OneLake DFS API	Four CSVs to Files/landing/*
6	Notebooks	deploy.py notebooks run · Jobs API RunNotebook	Create/update notebooks (fabricGitSource) and run 01 → 02 → 03 (→ 04)
7	Semantic model	deploy.py model · items API (format TMDL) + Power BI refreshes API	Deploy TMDL, reframe, validate with executeQueries
8	Report	deploy.py report · items API (PBIR)	Deploy pages, visuals and custom theme
9	Data agent	deploy.py agent · items API	Instructions + semantic-model data source, draft and published
10	Ontology	deploy.py run-nb4 ontology	Entity tables, then ontology definition (entity types, bindings, relationships)
11	Simulator	demo-control.ps1	Live bookings; latency measured
12	Verify	Playwright + DAX	Portal screenshots of every page, agent tests, KPI checks

15.1 Authentication

- All REST calls use tokens from `az account get-access-token` (scoped with `--subscription` or `--tenant` from config.json) for the audiences Fabric (`api.fabric.microsoft.com`), Power BI (`analysis.windows.net/powerbi/api`), OneLake storage (`storage.azure.com`) and SQL (`database.windows.net`).
- Every step is idempotent: items are looked up by type and name and updated with `updateDefinition` when they exist.

15.2 Repository structure

Path	Content
`config.json / .state.json`	Subscription, capacity, workspace name / IDs of deployed items and endpoints

Path	Content
`src/gen_data.py`	Synthetic data generator (hotels, reservations, guests, CRM, ERP, seats)
`src/load_pms.py, src/sqlconn.py`	PMS DDL + loader; Entra-token SQL connection helper
`src/fab.py, src/deploy.py`	Fabric REST helper (auth, LRO polling, definitions); step-based deployer
`src/notebooks.py`	Notebook sources 01–04
`src/model.py`	Semantic model: tables, columns, 53 measures, relationships, roles → TMDL
`src/report.py`	PBIR report builder (pages, visuals, theme)
`src/agent.py, src/ontology.py`	Data agent definition; ontology definition
`src/pms_live_sim.py, src/warmup.py, demo-control.ps1`	Live simulator, warm-up, demo control
`docs/`	DEMO_SCRIPT.md, screenshots, this documentation and its build scripts

16. Issues encountered and resolutions

Build log of problems found during implementation and testing, and how each was resolved.

#	Issue	Resolution
1	Package install via uv failed (TLS handshake)	Used pip with system Python; mssql-python wheel installed
2	No ODBC Driver 18 on the build machine	mssql-python (bundled driver) with access token in attrs_before (1256)
3	mssql-python rejected 'Connection Timeout' keyword	Removed the keyword from the connection string
4	bulkcopy opened a second connection without the token (login failed)	Automatic fallback to executemany batches of 20K
5	First forecast model ≈ 'same as last year'; Kos forecast to grow	Target = change vs LY; pace and seat features smoothed; monotonic constraints (MAE 3.31 vs 4.09)
6	Notebook 03 failed: indentation error at the start of a cell	Fixed; every cell is now compile-checked before deployment
7	Model deploy failed: measure 'Arrivals' clashes with column 'arrivals'	Renamed to 'Guest Arrivals' (names are case-insensitive in a table)
8	GOP margin unrealistic (56%)	Recalibrated SAP cost drivers → ~40%
9	vs LY measures showed −100% when LY was blank	DIVIDE(x − LY, LY) and IF(NOT ISBLANK(LY), …) patterns
10	Airline seats did not react to hotel filters	TREATAS(VALUES(dim_hotel[airport_code]), dim_airport[airport_code])
11	KPI comparison text clipped under cards	Tile = container textbox + two transparent cards
12	Channels & Guests visuals failed: duplicate guest_id on the one side	De-duplicated 99 Privilege Club IDs in notebook 02
13	Lakehouse SQL endpoint did not show new mirrored rows	Delta and Direct Lake were already current; SQL endpoint metadata sync is lazy for shortcuts → refreshMetadata in warm-up
14	Workspace folder names rejected ('·', '&', '.', '-')	Simple alphanumeric names (e.g. '4 Power BI')
15	Ontology lakehouse/graph could not be moved to a folder	Child items move with their parent ontology
16	Browser showed the previous report version after an update	Hard reload with a cache-busting query parameter
17	Live PMS page empty on a new day (TODAY() filter)	Rolling 24-hour window with NOW() and COALESCE(…, 0)
18	Ontology agent (preview) did not answer an automated prompt	Kept optional in the demo

17. Important considerations

17.1 Mirroring SQL Server (customer environment)

- SQL Server **2016–2022** mirroring uses **Change Data Capture**; SQL Server 2025 uses the Fabric change feed and requires Azure Arc.
- Tables **must have a primary key** (2016–2022). Legacy PMS tables without keys cannot be mirrored directly – mirror the existing data warehouse instead, or use an incremental Copy job from a readable replica.
- Mirroring runs only from the **primary replica** of an availability group; failover cluster instances are not supported.
- A **DDL change** on a mirrored table stops mirroring until CDC is disabled and re-enabled for that table (vendor upgrades!).
- Long-running transactions hold back transaction-log truncation – monitor log growth.
- Replication compute is free; mirrored storage is free up to a limit tied to capacity size; queries consume capacity. Up to 1,000 tables per mirrored database.
- Source **RLS, object-level permissions and dynamic data masking are not propagated** to OneLake – re-apply security in Fabric (semantic model RLS/OLS, OneLake security).

17.2 Direct Lake and the semantic model

- No scheduled refresh is needed; framing happens automatically on Delta commits (keep 'Keep Direct Lake data up to date' on). A manual reframe is useful after bulk rebuilds.
- Check the **Direct Lake guardrails** for the chosen SKU (rows per table, memory) when sizing production; exceeding them causes fallback or errors.
- Calculated tables and calculated columns are not used – all shaping happens in notebooks, which keeps the model Direct Lake-friendly.
- Measure names must be unique within a table regardless of case and must not collide with column names.
- `NOW()` / `TODAY()` evaluate in **UTC** in the service – align timestamps (the simulator writes UTC).

17.3 Security and governance

- RLS roles exist but have no members – assign GMs/regional managers before sharing; test with 'Test as role'.
- `dim_guest` contains personal data (names, emails). In production: hide or remove PII columns from broad models, apply object-level security, sensitivity labels and source-side masking.
- The data agent and Copilot respect the user's model permissions and RLS.

17.4 Capacity and licensing

- The whole demo runs on an **F8**; Copilot and data agents work from F2 upward.
- Below F64, report consumers need Power BI Pro licences; from F64, viewers with a free licence can consume content.
- Pause the capacity when the demo is not needed (a paused capacity makes the items unavailable).

17.5 Known limitations of the demo

- Data is synthetic and static apart from the live table; to roll the snapshot forward, change `AS_OF` and regenerate.
- Salesforce, SAP and airline data arrive as CSV files rather than live connectors.
- The forecast is trained on one season pair (2025 → 2026); production would use several years, weather/events and nightly retraining.
- The data agent takes 60–70 seconds per answer; pre-asked questions are kept in the chat as a fallback.

18. Appendices

18.1 Appendix A – Fabric item inventory

Workspace **Kyanos Hotels - Analytics & AI Demo**. Item IDs are generated per deployment and recorded locally in `.state.json`.

Folder	Type	Item
1 Sources PMS on SQL	SQLDatabase	KyanosPMS
1 Sources PMS on SQL	SQLEndpoint	KyanosPMS
2 OneLake unified data	Lakehouse	KyanosLakehouse
2 OneLake unified data	Notebook	01_Ingest_Salesforce_SAP_Airports
2 OneLake unified data	Notebook	02_Build_Gold_Model
2 OneLake unified data	SQLEndpoint	KyanosLakehouse
3 AI forecasting	MLExperiment	kyanos-occupancy-forecast
3 AI forecasting	MLModel	kyanos-occupancy-forecast
3 AI forecasting	Notebook	03_AI_Demand_Forecast_2027
4 Power BI	Report	Kyanos Hotels - Commercial Performance
4 Power BI	SemanticModel	Kyanos Hotels Model
5 Agents and Fabric IQ	DataAgent	Kyanos Revenue & Guest Assistant
5 Agents and Fabric IQ	GraphModel	KyanosHotelsOntology_graph_<id>
5 Agents and Fabric IQ	Lakehouse	KyanosHotelsOntology_lh_<id>
5 Agents and Fabric IQ	Notebook	04_Fabric_IQ_Ontology_Entities
5 Agents and Fabric IQ	Ontology	KyanosHotelsOntology
5 Agents and Fabric IQ	SQLEndpoint	KyanosHotelsOntology_lh_<id>

18.2 Appendix B – Semantic model column dictionary

18.2.1 dim_date (gold.dim_date)

Calendar 2025-2027. Snapshot (today) is 6 Oct 2026; is_closed_ytd = Jan-Sep closed months.

Column	Type	Visible	Notes
`date`	dateTime	yes	key (date table); format dd mmm yyyy
`year`	int64	yes	
`quarter`	string	yes	
`month_num`	int64	hidden	
`month`	string	yes	sort by month_num
`year_month`	string	yes	
`month_start`	dateTime	yes	format mmm yyyy
`week_start`	dateTime	yes	format dd mmm yyyy
`weekday_num`	int64	hidden	
`weekday`	string	yes	sort by weekday_num
`season`	string	yes	

Column	Type	Visible	Notes
is_actual	boolean	yes	
is_closed_ytd	boolean	yes	

18.2.2 dim_hotel (gold.dim_hotel)

The 24 Kyanos hotels and resorts with destination, airport, concept and general manager.

Column	Type	Visible	Notes
hotel_id	int64	hidden	
hotel_code	string	yes	
hotel_name	string	yes	
hotel_short_name	string	yes	
destination	string	yes	
region	string	yes	category StateOrProvince
airport_code	string	yes	
stars	int64	yes	
concept	string	yes	
rooms	int64	yes	
latitude	double	yes	category Latitude
longitude	double	yes	category Longitude
operating_model	string	yes	
general_manager	string	yes	
gm_email	string	yes	

18.2.3 dim_airport (gold.dim_airport)

Greek airports serving the hotels.

Column	Type	Visible	Notes
airport_code	string	yes	
airport_name	string	yes	
airport_region	string	yes	

18.2.4 dim_guest (gold.dim_guest)

Guest profile from the PMS enriched with Salesforce Privilege Club membership.

Column	Type	Visible	Notes
guest_id	int64	hidden	
guest_name	string	yes	
email	string	yes	
country_code	string	yes	
country	string	yes	category Country

Column	Type	Visible	Notes
language	string	yes	
member_id	string	yes	
is_member	boolean	yes	
loyalty_tier	string	yes	
marketing_opt_in	boolean	yes	
top_preference	string	yes	
lifetime_nights	int64	yes	

18.2.5 fact_hotel_day (gold.fact_hotel_day)

One row per hotel per day: rooms available vs sold and revenue. Future dates hold on-the-books business.

Column	Type	Visible	Notes
hotel_id	int64	hidden	
date	dateTime	hidden	
rooms_available	int64	hidden	
rooms_sold	int64	hidden	
room_revenue	double	hidden	
extras_revenue	double	hidden	
arrivals	int64	hidden	
is_actual	boolean	hidden	
total_revenue	double	hidden	

18.2.6 fact_hotel_cost (gold.fact_hotel_cost)

Monthly operating costs per hotel from SAP (payroll, F&B cost of sales, energy, commissions, maintenance, marketing).

Column	Type	Visible	Notes
hotel_id	int64	hidden	
month_start	dateTime	hidden	
cost_category	string	yes	
gl_account	string	yes	
amount_eur	double	hidden	

18.2.7 fact_reservation (gold.fact_reservation)

Every PMS reservation (incl. cancellations) with channel, segment, board, lead time and revenue.

Column	Type	Visible	Notes
reservation_id	int64	hidden	
confirmation_no	string	yes	
hotel_id	int64	hidden	

Column	Type	Visible	Notes
guest_id	int64	hidden	
room_type	string	yes	
booking_date	dateTime	yes	format dd mmm yyyy
arrival_date	dateTime	yes	format dd mmm yyyy
departure_date	dateTime	yes	format dd mmm yyyy
nights	int64	yes	
lead_time_days	int64	yes	
channel	string	yes	
channel_group	string	yes	
market_segment	string	yes	
tour_operator	string	yes	
board_type	string	yes	
adults	int64	yes	
children	int64	yes	
status	string	yes	
is_cancelled	boolean	hidden	
is_active	boolean	hidden	
rate_amount	double	hidden	
room_revenue	double	hidden	
extras_revenue	double	hidden	
cancel_date	dateTime	yes	format dd mmm yyyy
created_at	dateTime	hidden	

18.2.8 fact_guest_case (gold.fact_guest_case)

Salesforce Service Cloud guest cases (complaints, requests, compliments) with CSAT.

Column	Type	Visible	Notes
case_id	string	yes	
confirmation_no	string	yes	
hotel_id	int64	hidden	
created_date	dateTime	yes	format dd mmm yyyy
category	string	yes	
sub_category	string	yes	
origin	string	yes	
status	string	yes	
csat_score	int64	yes	

18.2.9 fact_airport_seats (gold.fact_airport_seats)

Weekly scheduled arriving airline seats per airport (2027 = published schedules).

Column	Type	Visible	Notes
airport_code	string	hidden	
week_start	dateTime	hidden	
arriving_seats	int64	hidden	
flights	int64	hidden	
season_year	int64	hidden	
data_status	string	yes	

18.2.10 fact_occupancy_forecast (gold.fact_occupancy_forecast)

AI occupancy forecast per hotel-day for Jan-Sep 2027 (booking pace + airline seats model, MLflow).

Column	Type	Visible	Notes
hotel_id	int64	hidden	
date	dateTime	hidden	
rooms_available	double	hidden	
otb_rooms	double	hidden	
ly_otb_rooms	double	hidden	
forecast_rooms_sold	double	hidden	
forecast_room_revenue	double	hidden	
ly_rooms_sold	double	hidden	
ly_room_revenue	double	hidden	
seat_growth	double	hidden	
pace_vs_ly	double	hidden	
otb_occ	double	hidden	
forecast_occ	double	hidden	
ly_occ	double	hidden	
model_run_id	string	hidden	
snapshot_date	dateTime	hidden	

18.2.11 live_pms_reservation (pms.Reservation)

LIVE: the PMS reservation table mirrored from SQL in near real time (no ETL).

Column	Type	Visible	Notes
ReservationId	int64	hidden	
ConfirmationNo	string	yes	
HotelId	int64	hidden	
GuestId	int64	hidden	

Column	Type	Visible	Notes
BookingDate	dateTime	yes	format dd mmm yyyy
ArrivalDate	dateTime	yes	format dd mmm yyyy
Nights	int64	yes	
Channel	string	yes	
MarketSegment	string	yes	
BoardType	string	yes	
RoomRevenue	decimal	hidden	
Status	string	yes	
CreatedAt	dateTime	yes	format dd mmm yyyy hh:nn:ss

18.3 Appendix C – Key code excerpts

18.3.1 fact_hotel_day – stay explosion and hotel calendar (notebook 02)

```
# fact_hotel_day: one row per hotel per calendar day (rooms available vs sold, revenue, arrivals)
act = spark.table("gold.fact_reservation").where("is_active")
nights = (act.withColumn("i", F.explode(F.sequence(F.lit(0), F.col("nights") - 1)))
    .select("hotel_id", F.date_add("arrival_date", F.col("i")).alias("date"),
            (F.col("room_revenue") / F.col("nights")).alias("rr"), (F.col("extras_revenue") /
F.col("nights")).alias("xr")))
sold = nights.groupBy("hotel_id", "date").agg(F.count("*").alias("rooms_sold"), F.sum("rr").alias("room_revenue"),
F.sum("xr").alias("extras_revenue"))
arr = act.groupBy("hotel_id", F.col("arrival_date").alias("date")).agg(F.count("*").alias("arrivals"))
cal = (spark.table("gold.dim_date").select("date").crossJoin(hotel.select(F.col("HotelId").alias("hotel_id"), "Rooms",
"OpenMMDD", "CloseMMDD", "Airport")))
md = F.date_format("date", "MM-dd")
cal = cal.withColumn("is_open", (md >= F.col("OpenMMDD")) & (md <= F.col("CloseMMDD")))
fact_day = (cal.join(sold, ["hotel_id", "date"], "left").join(arr, ["hotel_id", "date"], "left")
    .select("hotel_id", "date", F.when(F.col("is_open"), F.col("Rooms")).otherwise(0).alias("rooms_available"),
            F.coalesce("rooms_sold", F.lit(0)).alias("rooms_sold"), F.round(F.coalesce("room_revenue", F.lit(0.0)),
2).alias("room_revenue"),
            F.round(F.coalesce("extras_revenue", F.lit(0.0)), 2).alias("extras_revenue"), F.coalesce("arrivals",
F.lit(0)).alias("arrivals"),
            (F.col("date") <= F.lit(AS_OF)).alias("is_actual"))
    .withColumn("total_revenue", F.round(F.col("room_revenue") + F.col("extras_revenue"), 2)))
save(fact_day, "fact_hotel_day")
```

18.3.2 Rooms on the books at a snapshot date (notebook 03)

```
def otb_as_of(snap):
    """Rooms on the books per hotel-night as they stood on the snapshot date."""
    r = res[(res.booking_date <= snap) & ((res.cancel_date.isna()) | (res.cancel_date > snap)) & (res.status != "No-
Show")]
    r = r.loc[r.index.repeat(r.nights)].copy()
    r["date"] = r.arrival_date + pd.to_timedelta(r.groupby(level=0).cumcount(), unit="D")
    return r.groupby(["hotel_id", "date"]).size().rename("otb_rooms").reset_index()
```

18.3.3 Model training with MLflow (notebook 03)

```
mlflow.set_experiment("kyanos-occupancy-forecast")
params = dict(max_iter=300, learning_rate=0.04, max_leaf_nodes=15, min_samples_leaf=60, l2_regularization=1.0)
mask = np.random.default_rng(7).random(len(train)) < 0.8
with mlflow.start_run(run_name="pace-model-summer-2027") as run:
    model = HistGradientBoostingRegressor(**params, monotonic_cst=MONOTONIC, random_state=7)
    model.fit(train.loc[mask, FEATURES], train.loc[mask, TARGET])
    pred = np.clip(train.loc[~mask, "ly_occ"] + model.predict(train.loc[~mask, FEATURES]), 0, 1)
    mae = mean_absolute_error(train.loc[~mask, "occ"], pred)
    base = mean_absolute_error(train.loc[~mask, "occ"], train.loc[~mask, "ly_occ"])
    mlflow.log_params({**params, "features": ",".join(FEATURES), "target": TARGET, "snapshot": str(AS_OF.date())})
    mlflow.log_metric("mae_occupancy_pts", round(mae * 100, 2))
    mlflow.log_metric("naive_same_as_last_year_mae_pts", round(base * 100, 2))
    model.fit(train[FEATURES], train[TARGET])
    mlflow.sklearn.log_model(model, "model", registered_model_name="kyanos-occupancy-forecast",
input_example=train[FEATURES].head(5))
print(f"Holdout MAE {mae*100:.1f} occupancy points vs. naive 'same as last year' {base*100:.1f}")
```

18.3.4 Semantic model deployment and refresh (src/deploy.py)

```
state["model"] = upsert("SemanticModel", "Kyanos Hotels Model", MD.model_parts(WS, state["lh"]), "TMDL")
POST https://api.powerbi.com/v1.0/myorg/groups/{ws}/datasets/{model}/refreshes   {"type": "full"}
POST https://api.powerbi.com/v1.0/myorg/groups/{ws}/datasets/{model}/executeQueries {"queries":[{"query":"EVALUATE ..."}]}
```

18.4 Appendix D – Glossary

Term	Meaning
ADR	Average daily rate = room revenue ÷ rooms sold
RevPAR	Revenue per available room = room revenue ÷ rooms available (= ADR × occupancy)
TRevPAR	Total revenue (rooms + F&B + other) ÷ rooms available
GOP / GOPPAR	Gross operating profit = total revenue − operating costs; GOP per available room
OTB / pace	On-the-books business for future dates / OTB compared with the same point last year
LY	Last year, same period (SAMEPERIODLASTYEAR)
Mirroring	Fabric feature that continuously replicates a database into OneLake as Delta
Shortcut	OneLake pointer to data in another item or store, without copying
Direct Lake	Power BI storage mode that reads Delta tables in OneLake directly, without import refresh
Framing	Direct Lake step that points the model at the latest Delta version
TMDL / PBIR	Text-based definition formats for semantic models / Power BI reports
MLflow	Experiment tracking and model registry built into Fabric Data Science
Data agent	Fabric item that answers natural-language questions over selected data sources
Ontology (Fabric IQ)	Business entities and relationships bound to OneLake data, navigable as a graph